Cloud Native Data Engineering

Week 7. NoSQL: Document, Key-Value and Graph Databases

Ashish Pujari | apujari@uchicago.edu

Lecture Outline

1. NoSQL Overview
2. Document Data Modeling
3. Key-Value Stores: DynamoDB
4. Graph Data Modeling
5. Access Patterns and Trade-offs



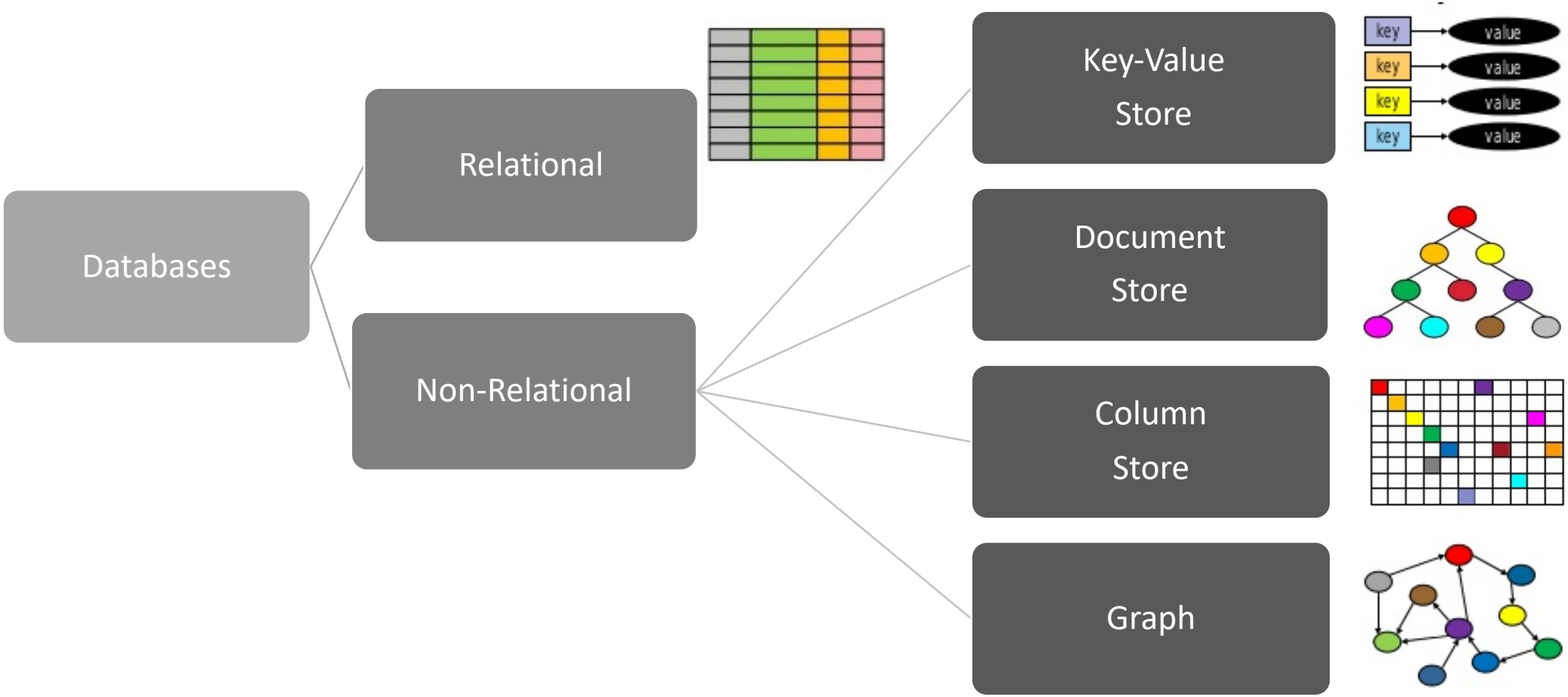
NoSQL Databases Overview

Module 1

Why NoSQL?

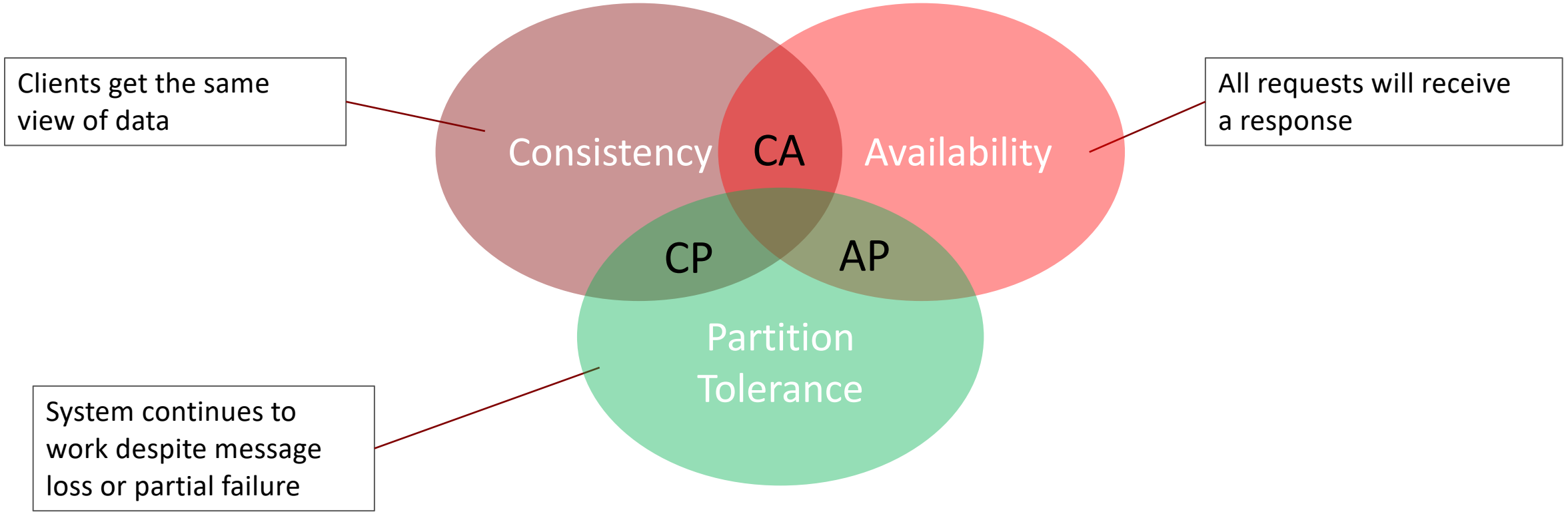
- Relational databases struggle with: massive scale, flexible schemas, semi-structured data
- NoSQL: "Not Only SQL" — broad category of non-relational databases
- Design goals: horizontal scalability, flexible schema, high availability
- Trade-off: sacrifice some consistency guarantees (ACID → BASE) for scale
- NoSQL does not replace relational — both have appropriate use cases

Database Types - Structure



CAP Theorem

- In any distributed system, you can only guarantee 2 of the 3 properties at once — pick your pair



ACID vs BASE

- ACID (relational): Atomicity, Consistency, Isolation, Durability — strict guarantees
- BASE (NoSQL): Basically Available, Soft state, Eventual consistency
- Eventual consistency: given enough time, all replicas converge to the same state
- Strong consistency possible in some NoSQL systems (Spanner, Cosmos with strong level)
- Choose consistency model based on business requirements, not database default

Key-Value Stores

- Simplest NoSQL model: map a unique key to a value (string, binary, or structured)
- O(1) get and set by key; no query language; no secondary indexes by default
- Redis: in-memory; sub-millisecond latency; rich data structures (list, set, hash, sorted set)
- DynamoDB: serverless; auto-scaling; single-digit millisecond reads; massive scale
- Use cases: session caching, rate limiting, leaderboards, feature flags, pub/sub



Key	Value
1001	Ash , Algebra, A
1002	Jeff , Physics, B
1003	Judy , English, A
1004	Ram , Spanish, C

Wide-Column Stores

- Each row has a row key; columns are dynamic and can differ between rows
- Optimized for very high write throughput and wide rows with sparse columns
- Bigtable (GCP): petabyte-scale; < 10ms reads; backs Google Search, Maps, Analytics
- Cassandra: open-source; tunable consistency; no single point of failure; AP by default
- Use cases: time-series (IoT sensor readings), activity logs, recommendation signals

Column1	Column2	Column3	Column4
1001	1002	1003	1004
Ash	Jeff	Judy	Ram
Algebra	Physics	English	Spanish
A	B	A	C

NoSQL Database Products

Database Type	Database Names
Key-Value	redis, amazon DynamoDB, riak, MEMCACHED
Columnar	APACHE HBASE, Cassandra, Google BigQuery
Document	mongoDB, amazon DynamoDB, Google Cloud Datastore, Azure Cosmos DB, CouchDB relax
Graph	neo4j, TITAN, OrientDB, INFINITEGRAPH



Document Data Modeling

Module 2

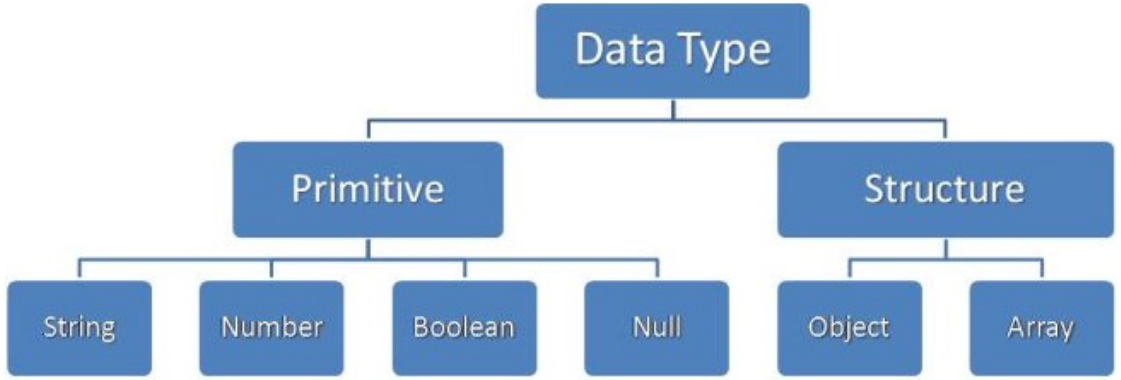
JSON Overview

- Lightweight data-interchange format
- Platform independent
- Built on two structures :
  - Collection of name/value pairs
  - An ordered list of values

```
{
  "message": {
  "date": "07-04-2016",
  "hour": "08:30",
  "to": "Receiver",
  "from": "Sender",
  "body": "This is an email message"
}}
```

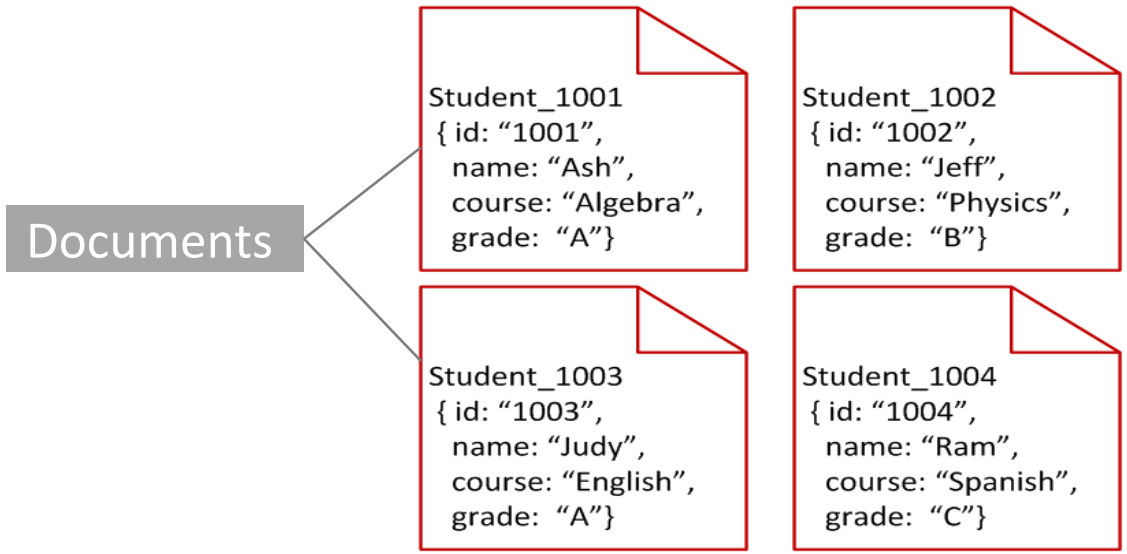
JSON Data Types

- A number (integer or floating point)
- A string (in double quotes)
- A Boolean (true or false)
- An array (in square brackets)
- An object (in curly braces)
- null



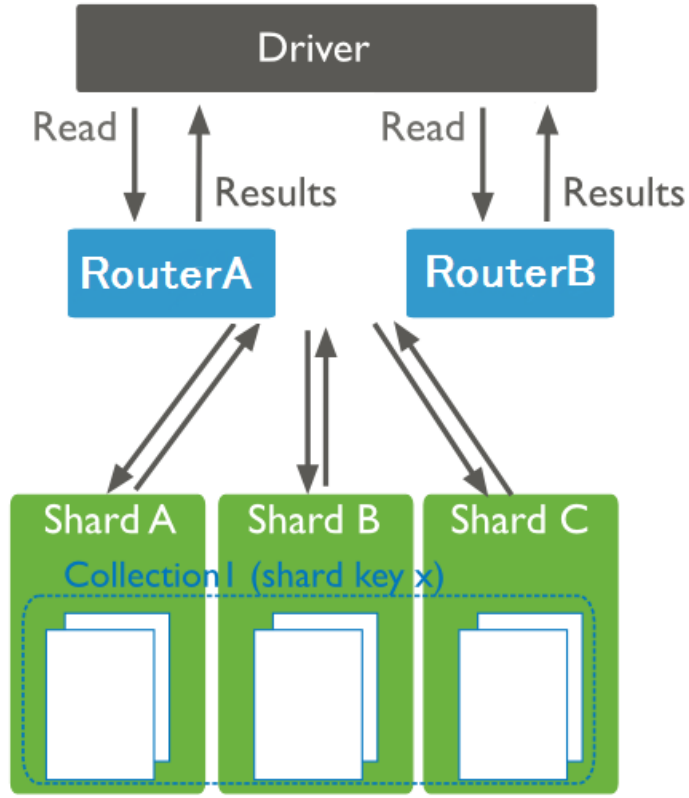
Document Database Concepts

- Documents: self-describing JSON/BSON objects with flexible per-document schema
- Collection: group of documents; analogous to a table but schema is per-document
- No joins at query time — related data is embedded or referenced
- Schema-less ≠ schema-free: enforce structure via validation rules or application layer
- MongoDB Atlas, GCP Firestore, AWS DocumentDB, CouchDB are leading platforms

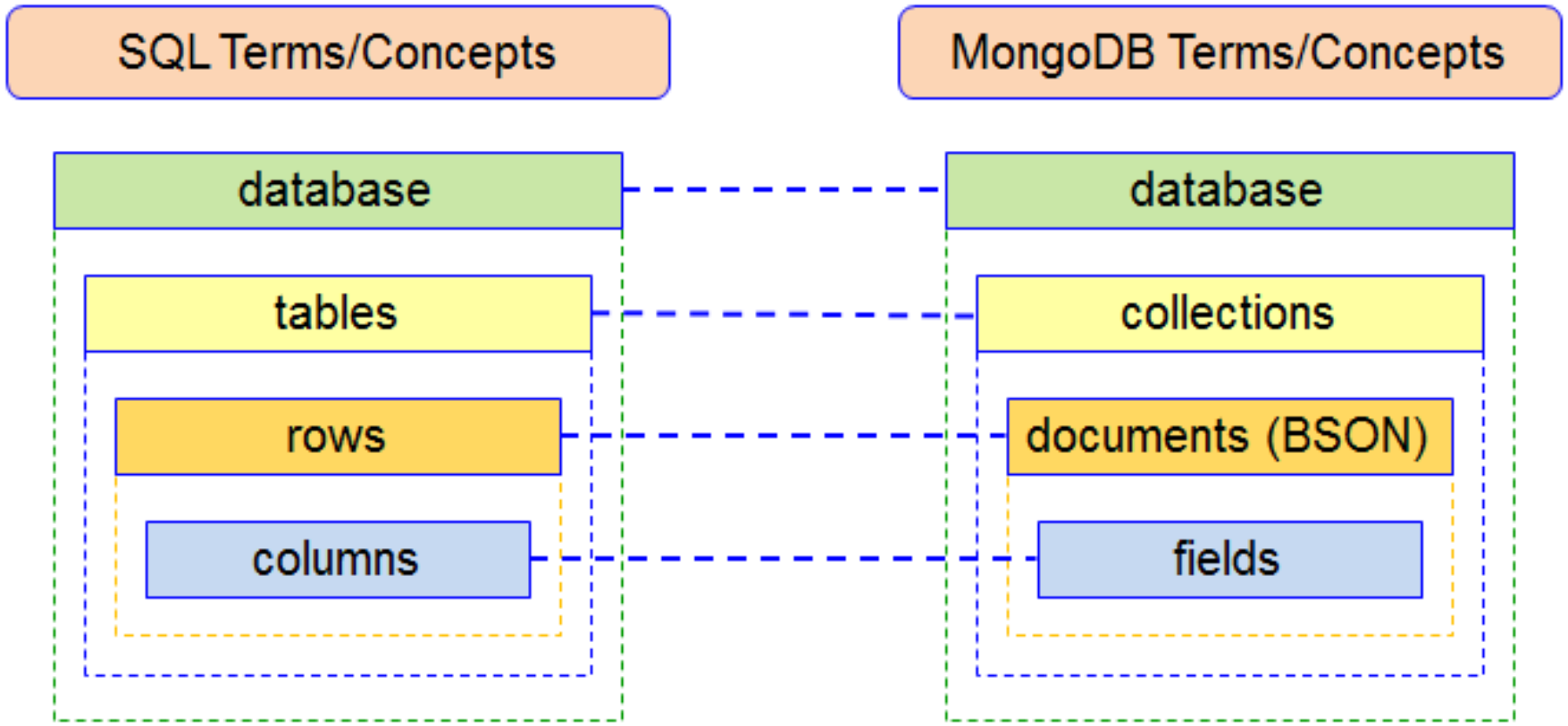


MongoDB Architecture

- Background
  - Open-source Document Oriented database
  - Developed by 10gen (2007)
  - Written in C++
- Features
  - Scale-out architecture
  - On-prem or on cloud
  - Scale reads and writes along with several nodes
  - Supports APIs (drivers) in: JavaScript, Python, Ruby, Perl, Java, JavaScala, C#, C++, Haskell, Erlang



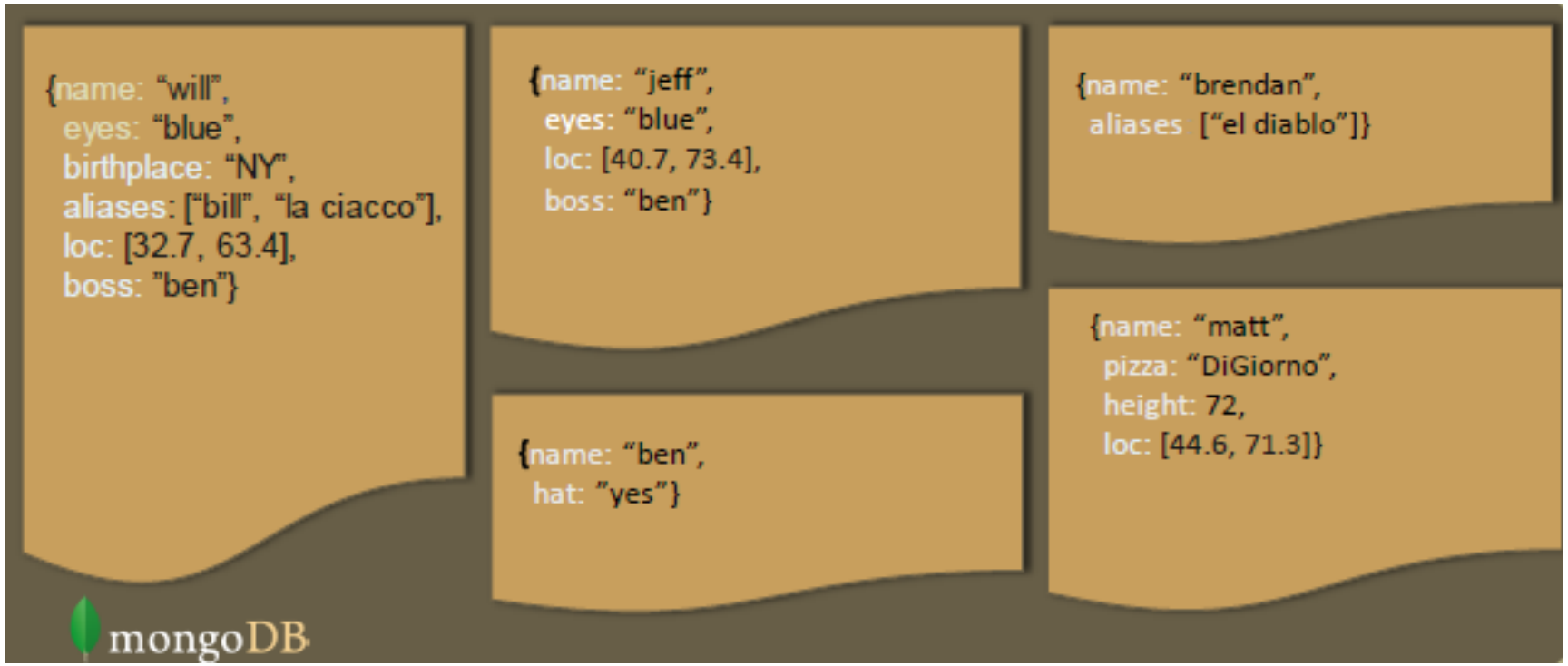
MongoDB Concepts



http://sql-vs-nosql.blogspot.com

Schema-Free Documents

- MongoDB does not need any pre-defined data schema
- Every document in a collection could have different data



BSON: Binary JSON

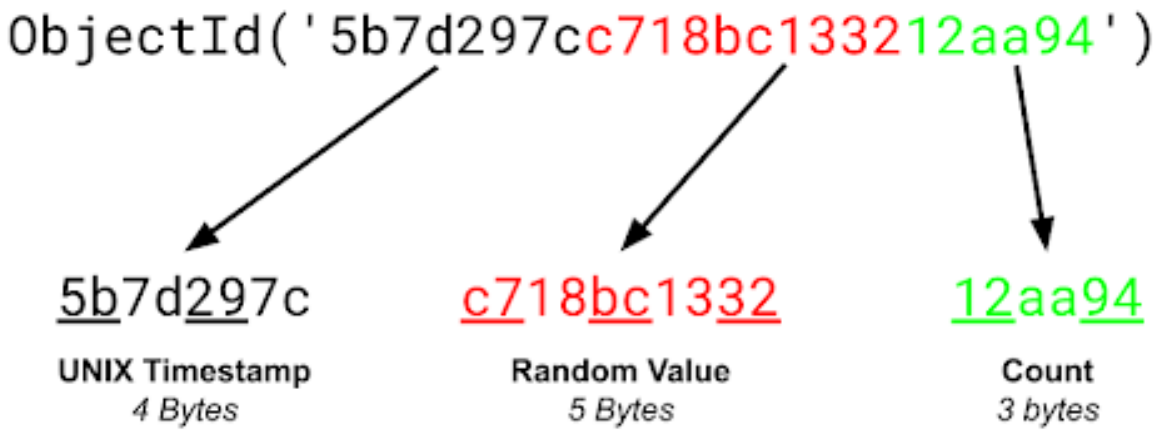
- MongoDB transfers and stores data in BSON
- Lightweight, traversable & efficient format
- MongoDB drivers convert to appropriate data types

```
// JSON
   { "hello" : "world" }

// BSON
"\x16\x00\x00\x00\x02hello\x00
\x06\x00\x00\x00world\x00\x00"
```

MongoDB ObjectId

- _id is a unique identifier or a primary key in MongoDB
- _id has an ObjectId datatype



Embedded vs Referenced Documents

```
{
  _id: <ObjectId1>,
  username: "123xyz",
  contact: {
             phone: "123-456-7890",
             email: "xyz@example.com"
           },
  access: {
             level: 5,
             group: "dev"
           }
}
```

Embedded sub-document

Embedded sub-document

user document

```
{
  _id: <ObjectId1>,
  username: "123xyz"
}
```

contact document

```
{
  _id: <ObjectId2>,
  user_id: <ObjectId1>,
  phone: "123-456-7890",
  email: "xyz@example.com"
}
```

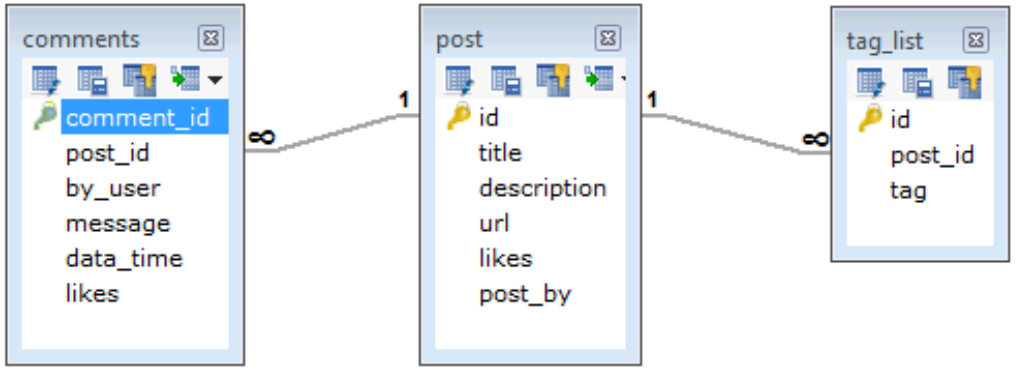
access document

```
{
  _id: <ObjectId3>,
  user_id: <ObjectId1>,
  level: 5,
  group: "dev"
}
```

https://docs.mongodb.com/manual/core/data-modeling-introduction/#flexible-schema

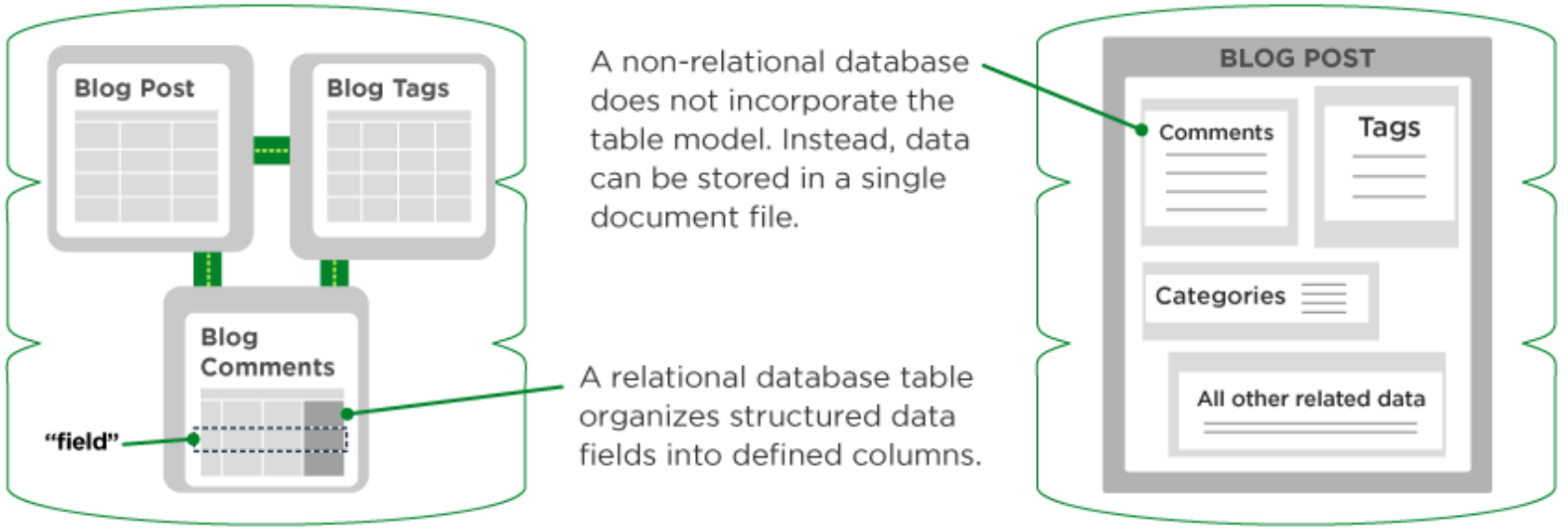
Blog Data Model: Relational

- Every post has a title, description, and URL
- One or more tags per post
- Publisher name and total likes
- Comments with user, message, time, likes
- Zero or more comments per post
- Relational design needs at least three tables



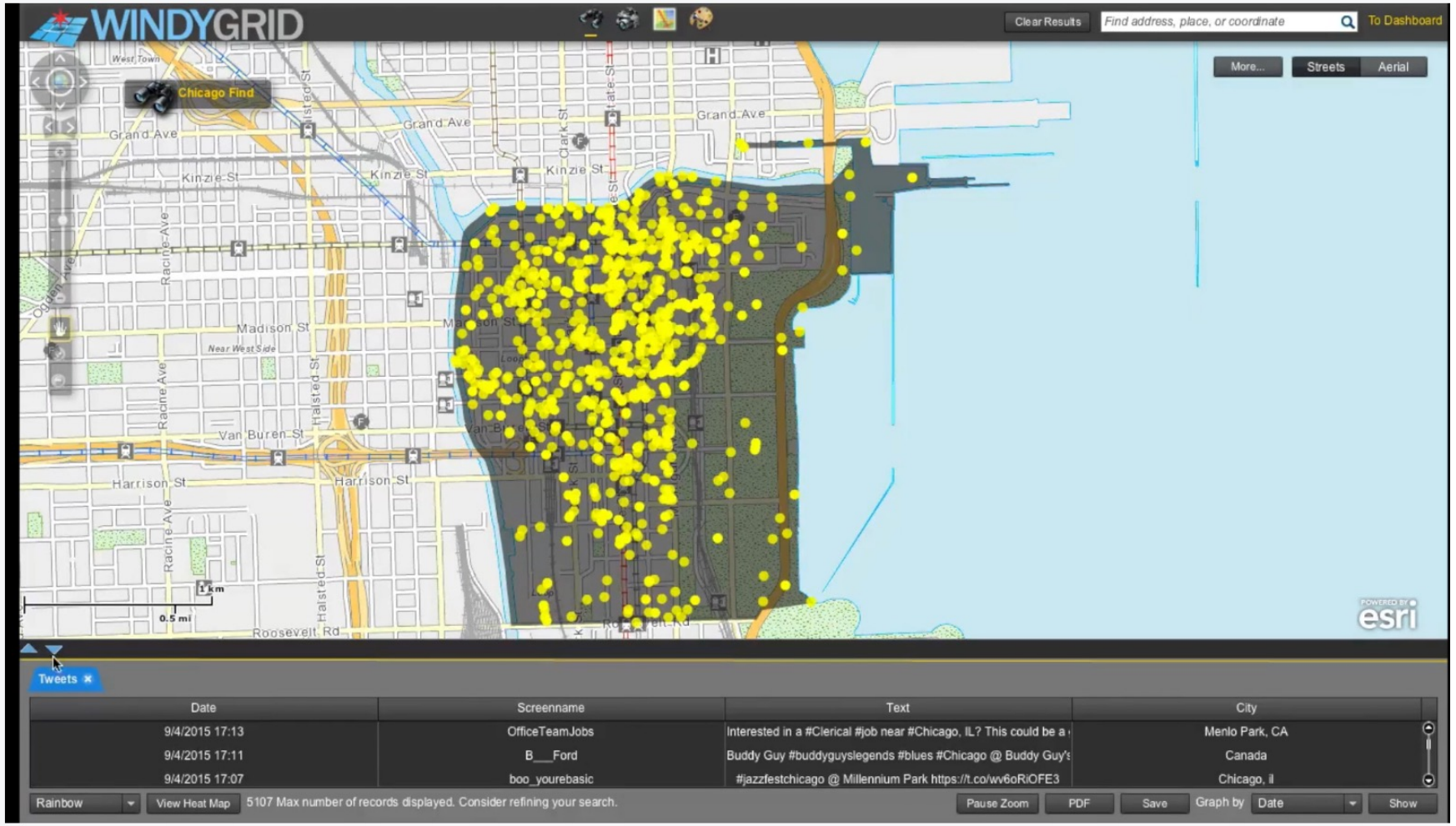
Blog Data Model: Document

- One post collection holds tags and comments in a single document




```
{
   _id: POST_ID
   title: TITLE_OF_POST,
   description: POST_DESCRIPTION,
   by: POST_BY,
   url: URL_OF_POST,
   tags: [TAG1, TAG2, TAG3],
   likes: TOTAL_LIKES,
   comments: [
      {
         user:'COMMENT_BY',
         message: TEXT,
         dateCreated: DATE_TIME,
         like: LIKES
      },
      {
         user:'COMMENT_BY',
         message: TEXT,
         dateCreated: DATE_TIME,
         like: LIKES
      }
   ]
}
```

MongoDB Use Case: City of Chicago



https://www.mongodb.com/customers/city-of-chicago



Key-Value Stores: DynamoDB

Module 3

DynamoDB Overview

- Fully managed, serverless key-value store
  - No servers, patching, or capacity planning
- Items are looked up by primary key
  - Single-digit millisecond reads at any scale
- Also stores document-style items
  - Nested lists and maps inside an item
- No joins and no ad hoc SQL
  - Design the table around access patterns

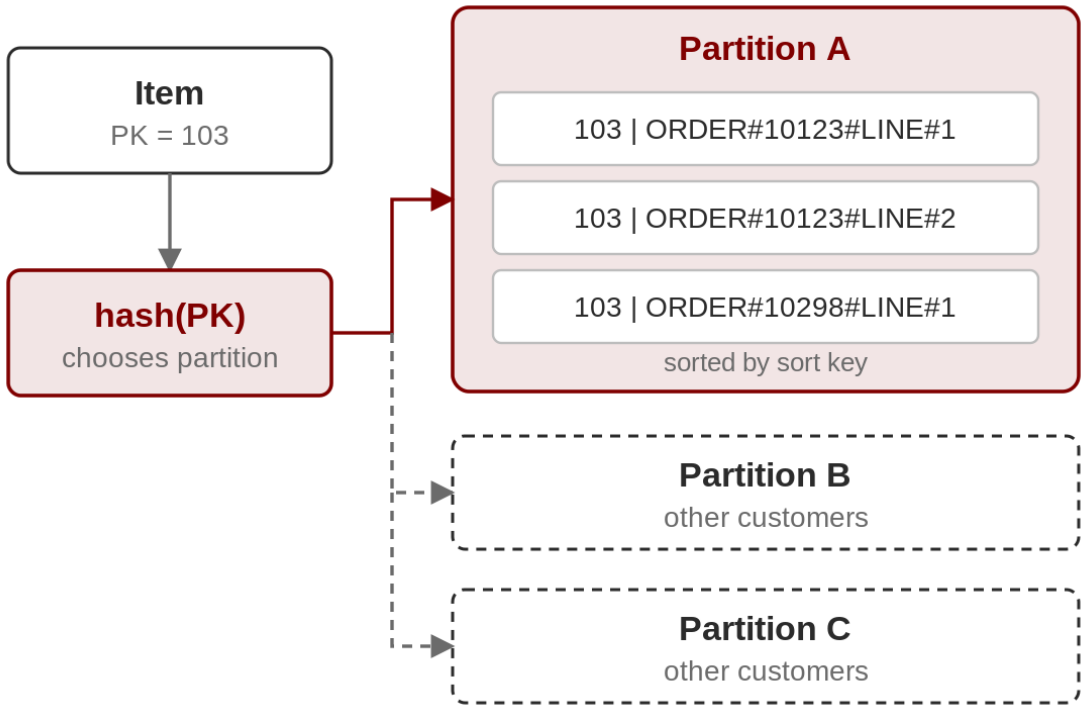
ClassicModels_Orders (illustrative items)

customerNumber Partition key	**SK** Sort key	**Other attributes** No fixed schema
103	ORDER#10123#LINE#1	orderDate, status, comments productCode, quantityOrdered
103	ORDER#10123#LINE#2	orderDate, status productCode, priceEach
112	ORDER#10124#LINE#1	orderDate, status productCode, quantityOrdered

Null values are omitted; each item carries only its own attributes

Partition Keys and Sort Keys

- Partition key chooses the partition
  - DynamoDB hashes it to a storage location
- Sort key orders items in a partition
  - Enables range queries on the key
- Choose high-cardinality partition keys
  - Avoids hot partitions
- Primary key must be unique per item
  - Partition key alone, or partition plus sort key



A query on PK = 103 reads one partition

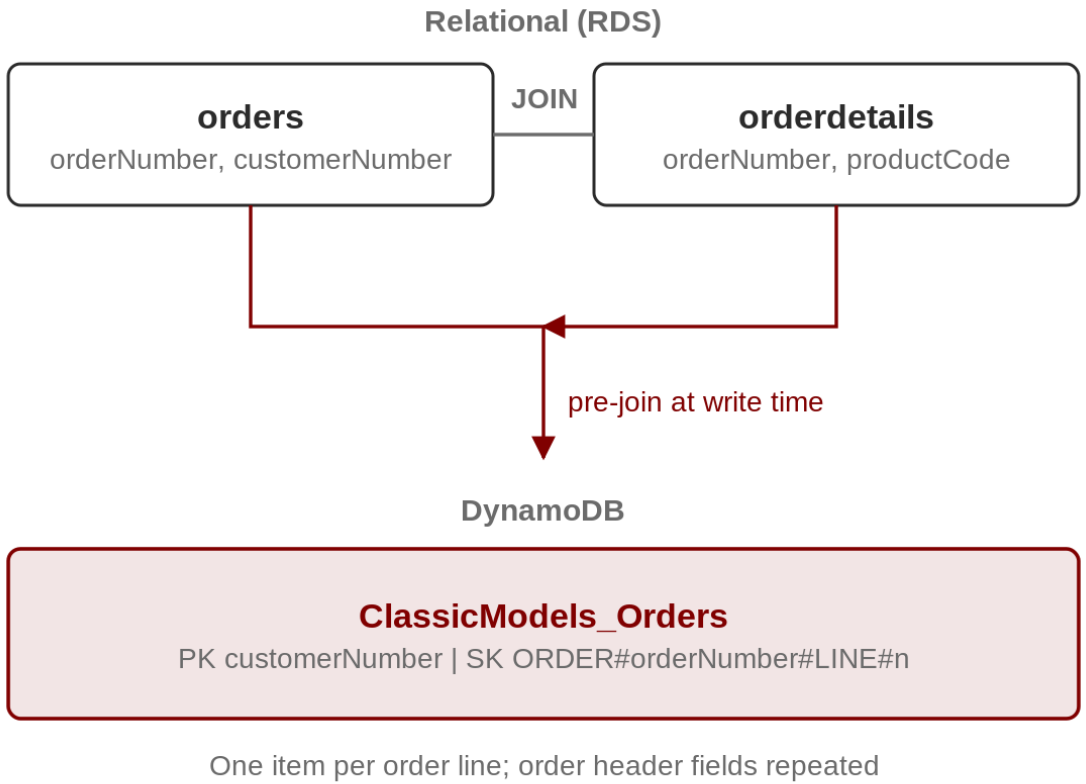
Read Operations

- Cost depends on how much of the table is read

Operation	What it reads	Cost	SQL equivalent
get_item	One item by full primary key	Lowest: constant time	WHERE pk = 103 AND sk = ...
query	Items sharing a partition key, optional sort key range	Low: one partition	WHERE customerNumber = 103
scan	Every item, then filters	High: reads the whole table	Full table scan with WHERE

Denormalized Table Design

- Relational splits data and joins at read
  - orders and orderdetails need a JOIN
- DynamoDB pre-joins at write time
  - One item per order line with header fields
- One query returns a customer's orders
  - Sorted by the sort key, no JOIN



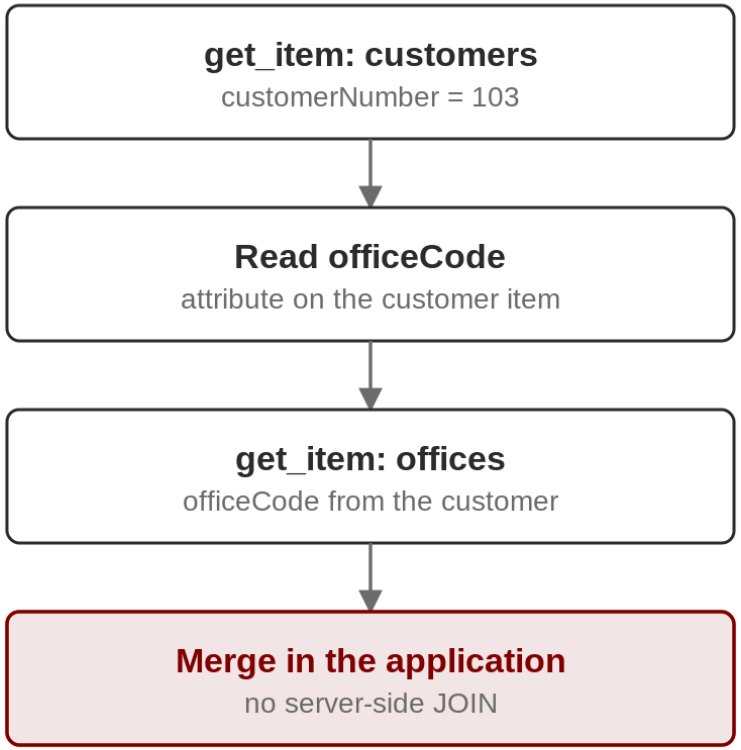
Secondary Indexes

- Query on non-key attributes needs an index, or it becomes a scan
  - Pre-compute attributes at write time to index them

	Global secondary index	Local secondary index
Keys	Different partition and sort key	Same partition key, different sort key
Created	Any time	At table creation only
Reads	Eventually consistent only	Eventual or strong
Capacity	Own throughput and storage cost	Shares the table's capacity

Flexible Schema and Application Joins

- Add attributes to a single item
  - No ALTER TABLE; other items unchanged
- No server-side joins
  - Relate items with key lookups
- Join in application code
  - Constant-time reads on each side



Each lookup is a constant-time key read

DynamoDB Patterns

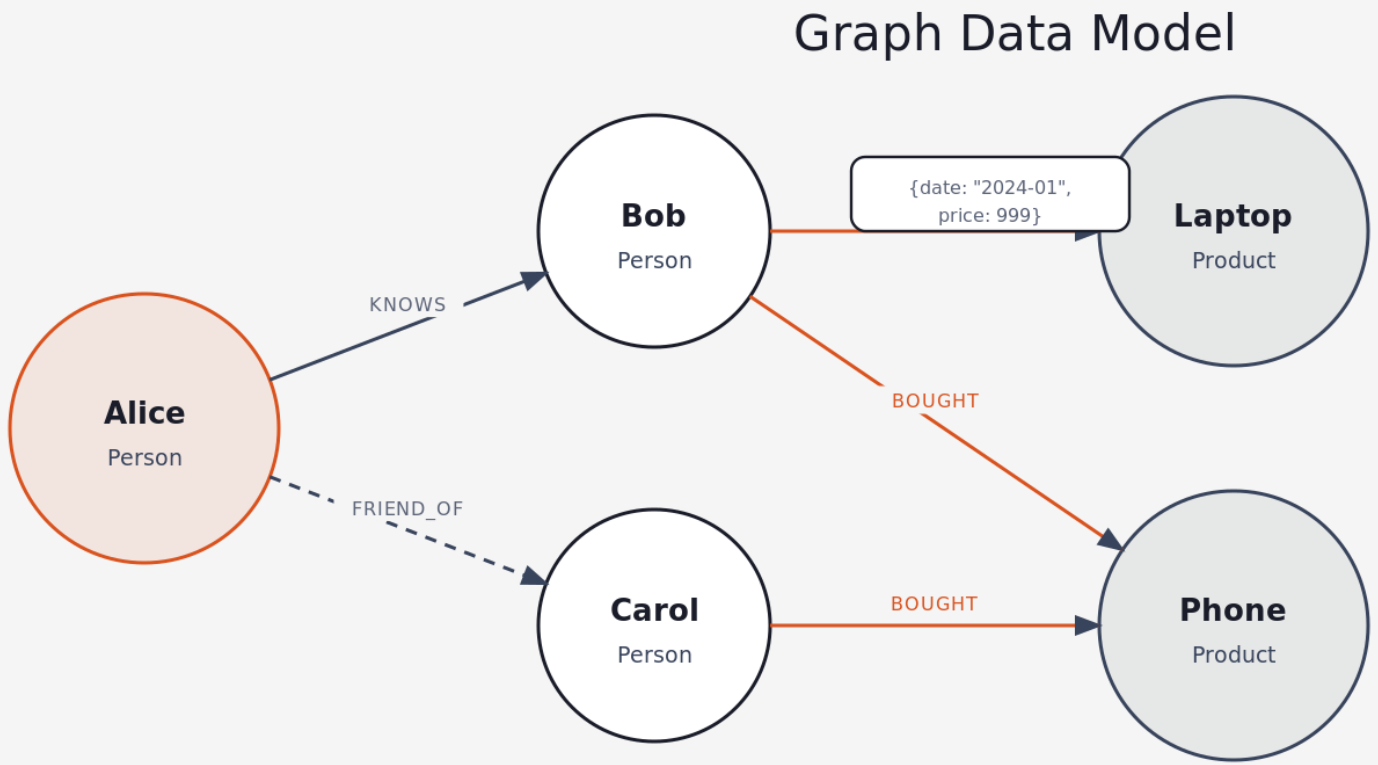
- Single-table design: store all entity types in one table; differentiate by pk/sk prefix
- GSI (Global Secondary Index): alternate partition + sort key for different access patterns
- LSI (Local Secondary Index): alternate sort key within the same partition
- TTL: automatically expire items after a timestamp attribute; free background deletion
- DynamoDB Streams: ordered log of item changes; feed Lambda or Kinesis for CDC



Graph Data Modeling

Module 4

Graph Data Model



Cypher Query

```
MATCH (p:Person)-
[:BOUGHT]->
(prod:Product)
WHERE p.name = 'Bob'
RETURN prod.name
```

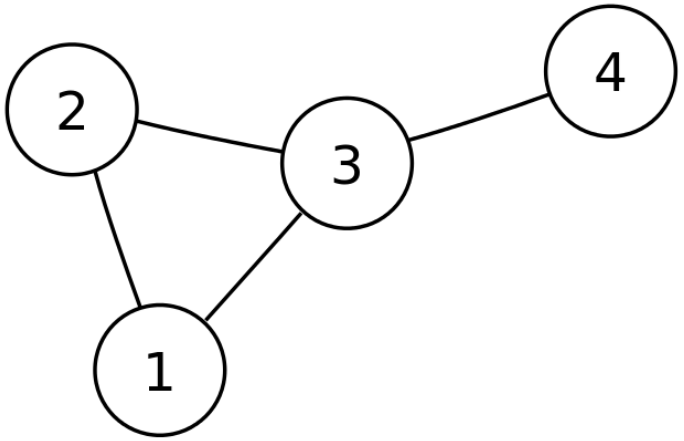
→ ['Laptop', 'Phone']

Neo4j / AuraDB
pattern matching
traversal in O(depth)
vs O(n) table scan

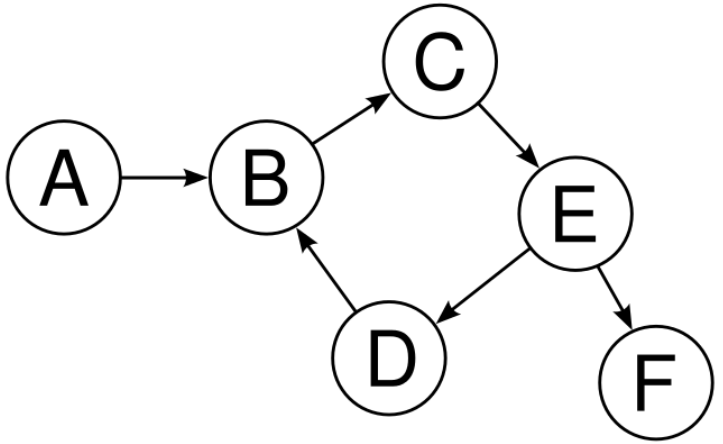
Graphs excel at connected data: recommendations, fraud rings, supply chains, social networks — relationships are first-class citizens

Directed and Undirected Graphs

Undirected Graph

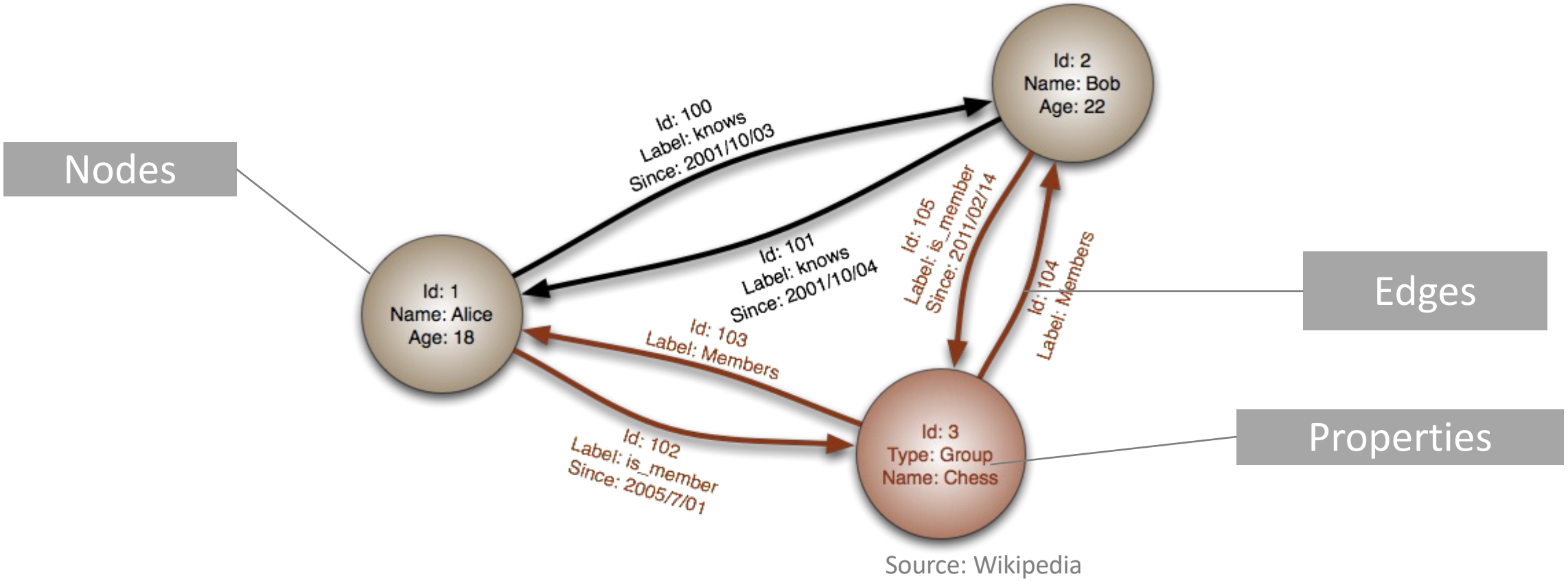


Directed Graph



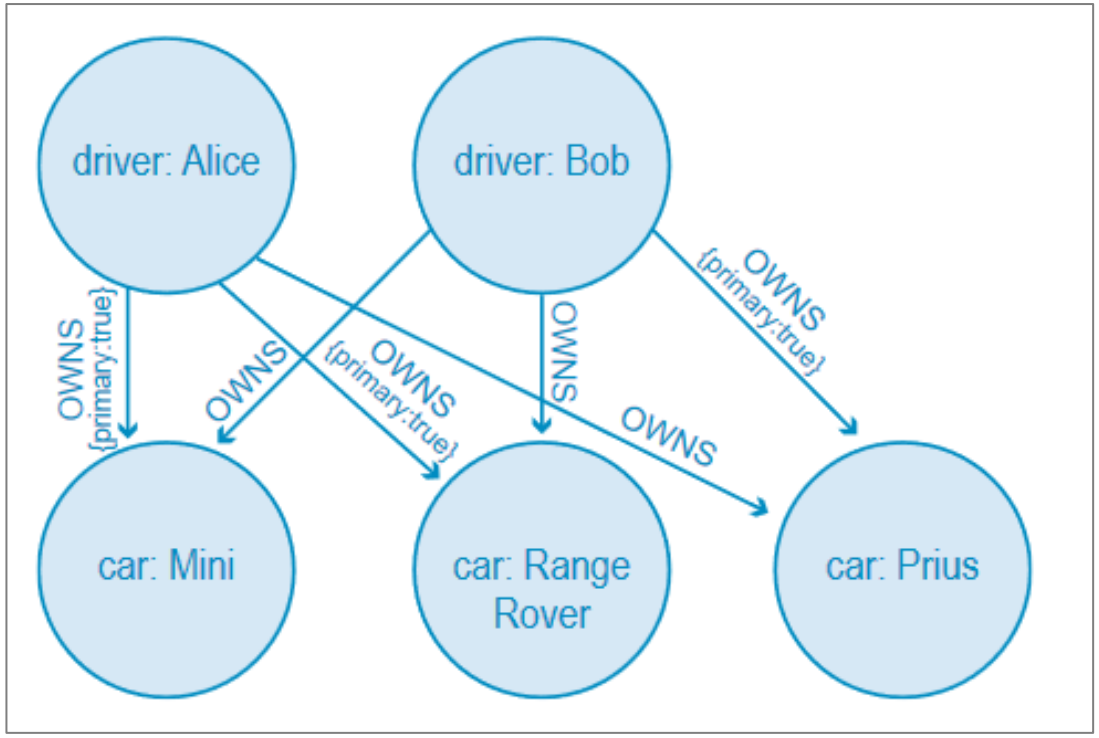
Sets	Undirected Graph	Directed Graph
Vertices	{1, 2, 3, 4}	{A, B, C, D, E, F}
Edges	{(1, 2), (2, 1), (2, 3), (3, 2), (1, 3), (3, 1), (3, 4), (4, 3)}	{(A,B),(B,C),(C,E),(E,D),(D,E),(E,F)}

Property Graph Example

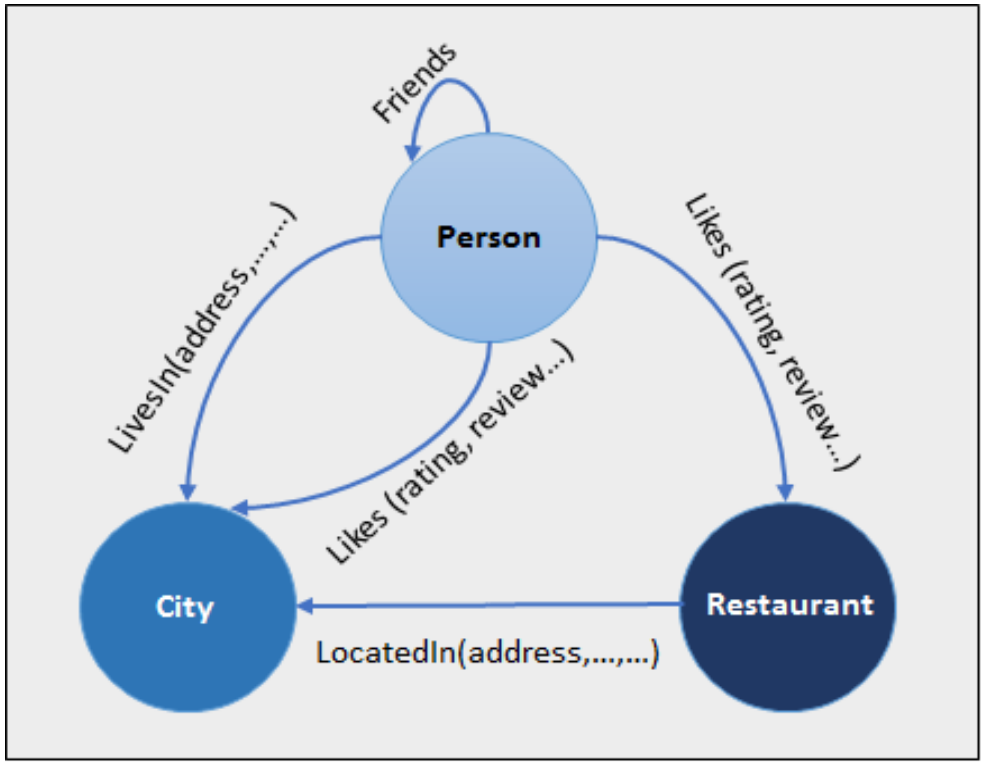


Source: Wikipedia

Graph Example: Ownership and Ratings

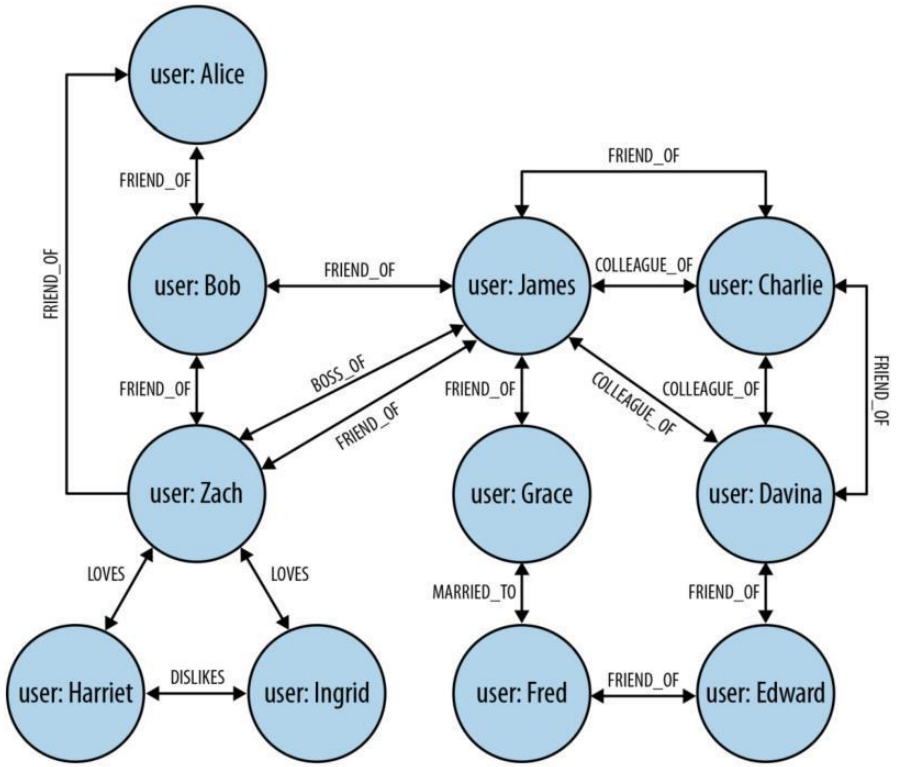


Car Ownership Graph

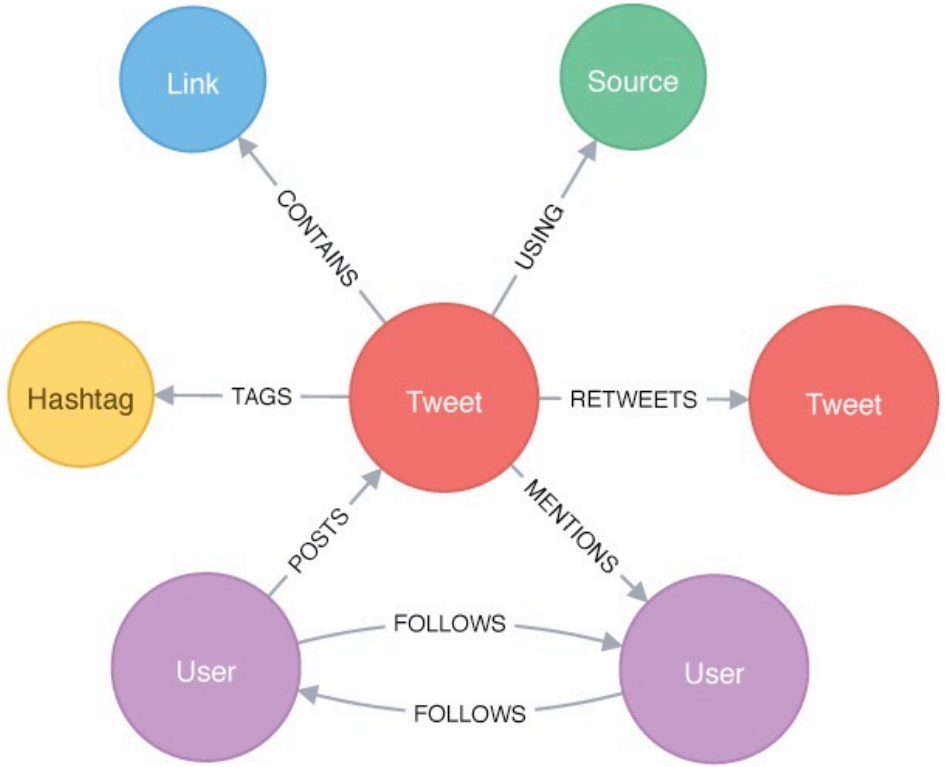


Restaurant, Person Graph

Graph Example: Social Networks

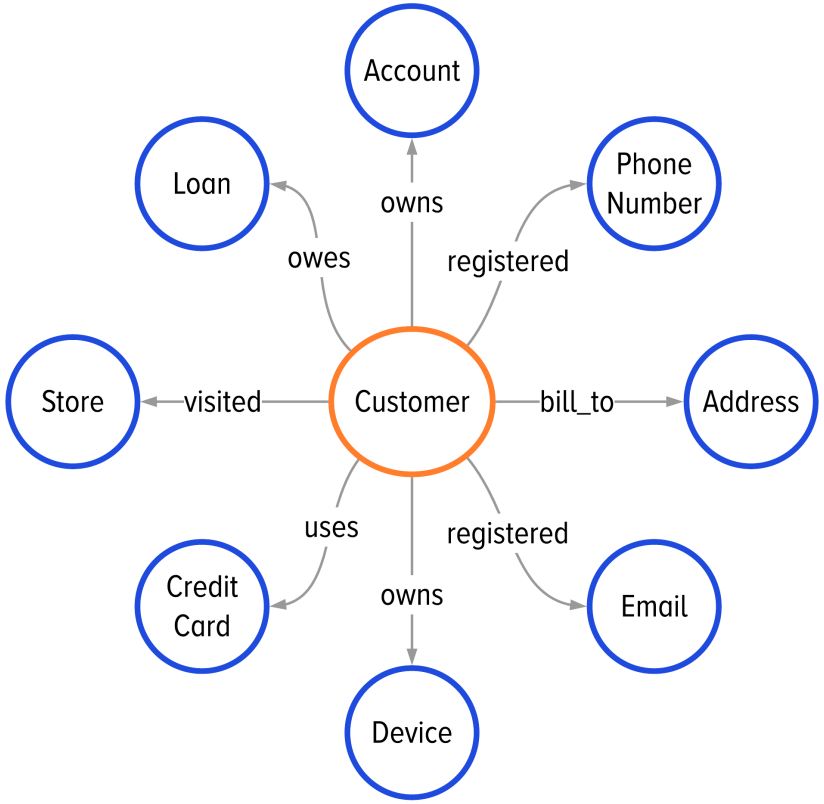


Social Network Graph

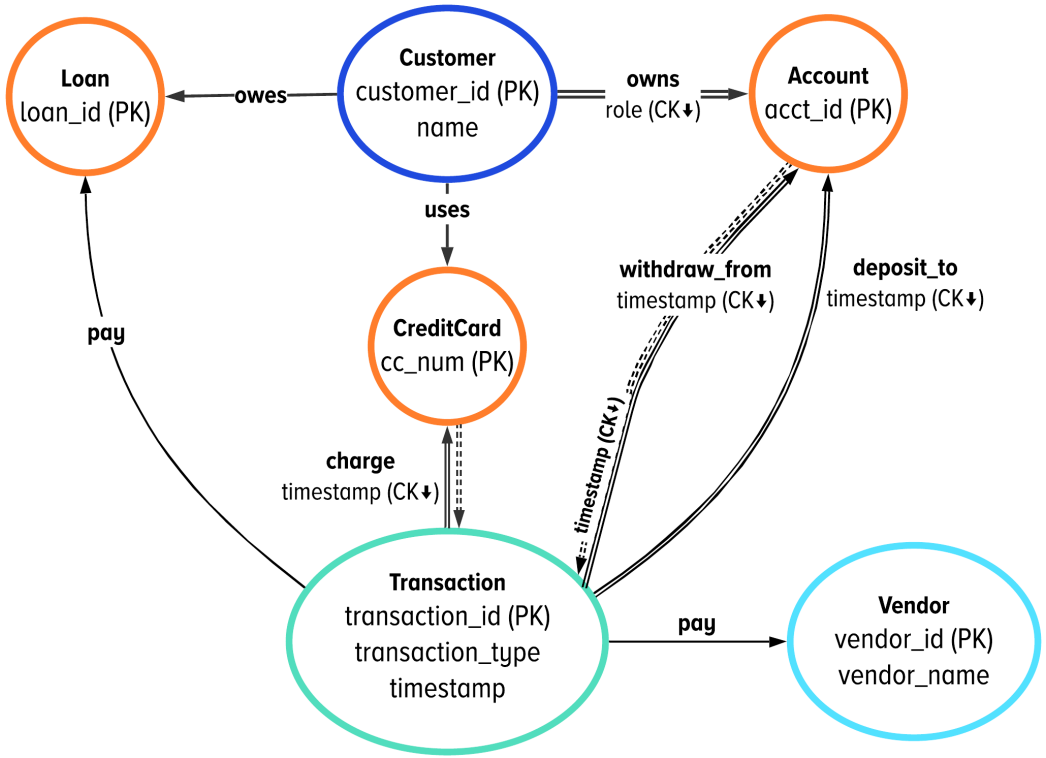


Twitter Graph

Graph Example: Financial



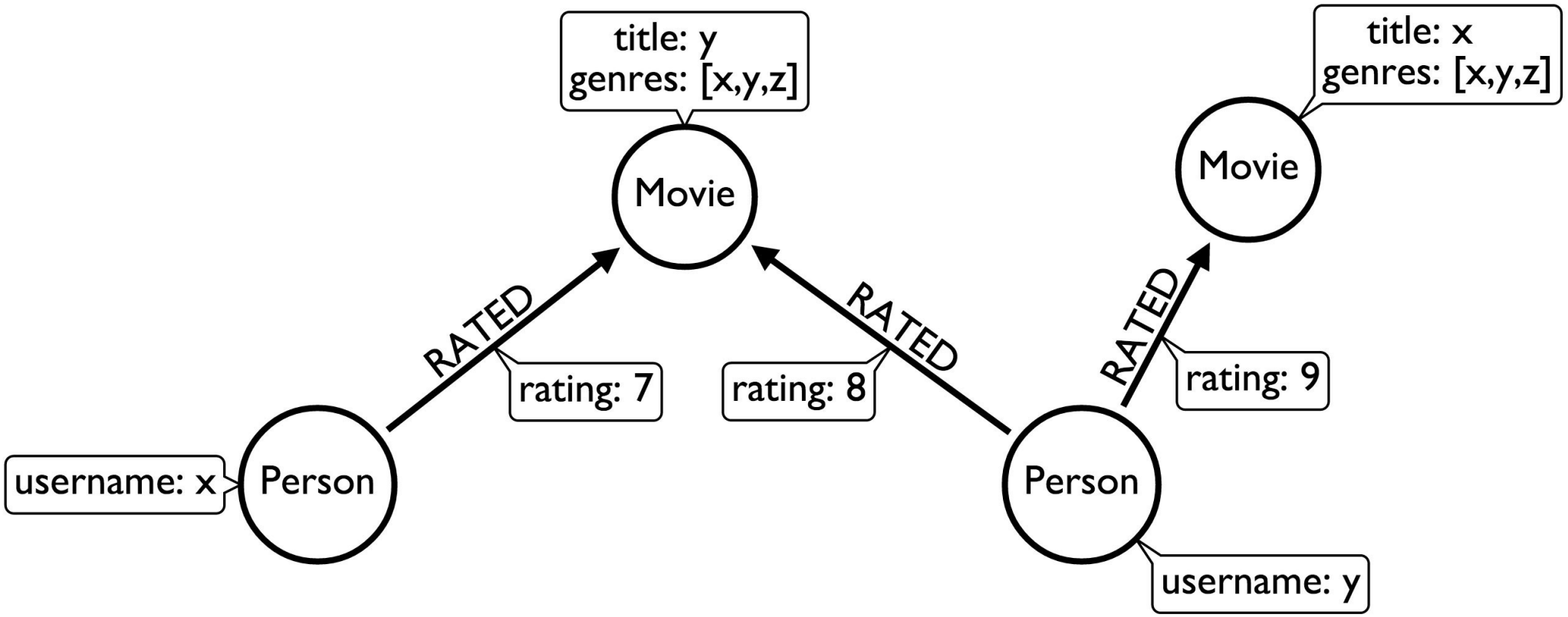
Customer Graph



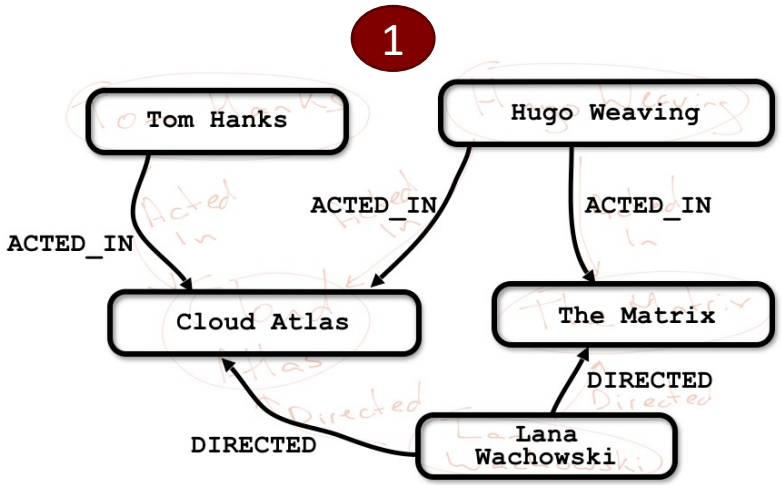
Transaction Graph

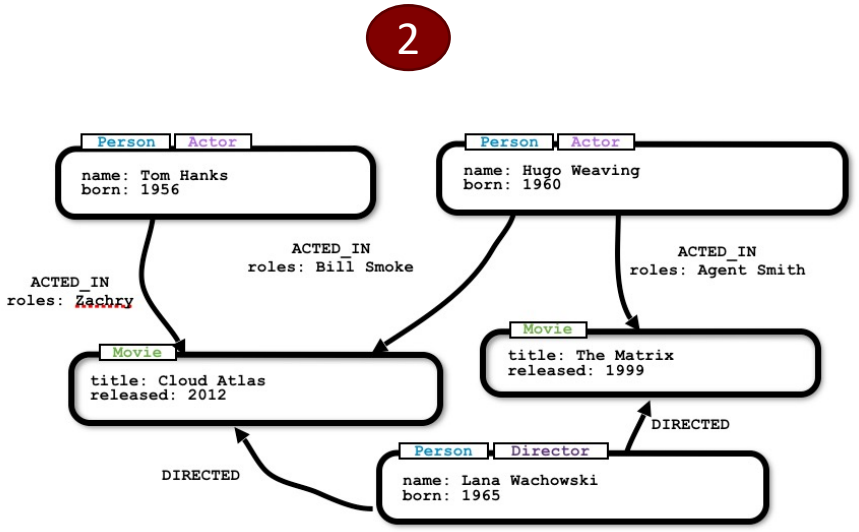
[The Practitioner's Guide to Graph Data]

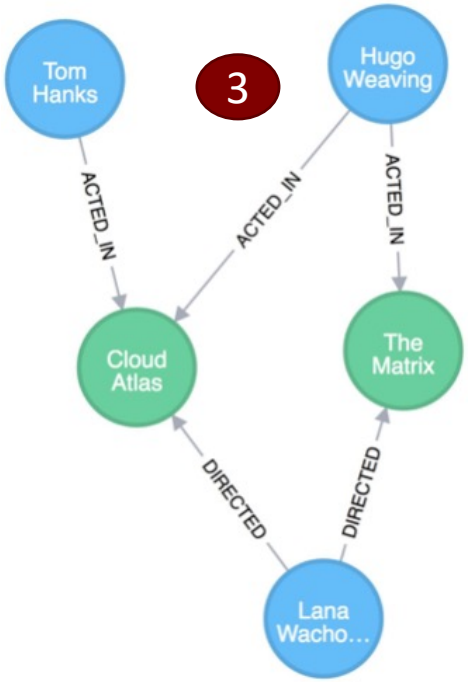
Graph Example: Movies



Graph Modeling Steps

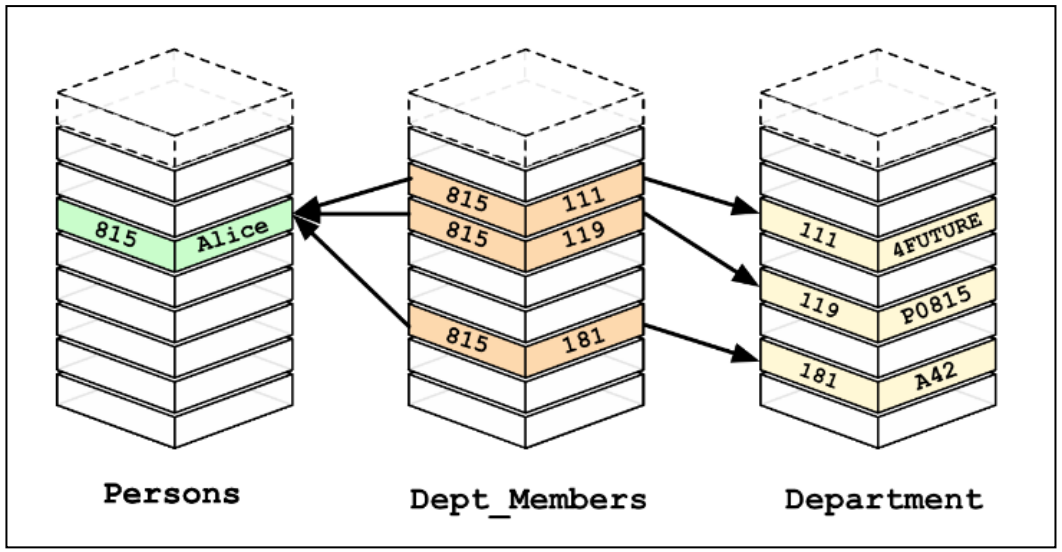


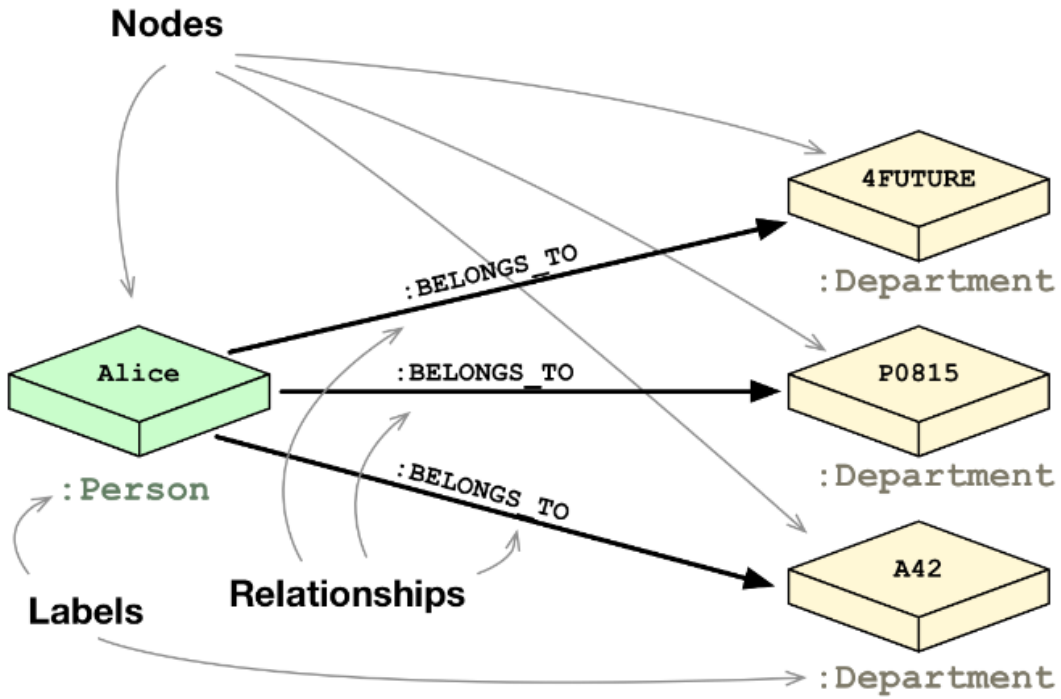




https://neo4j.com/developer/guide-data-modeling/

Relational vs Graph Model Example

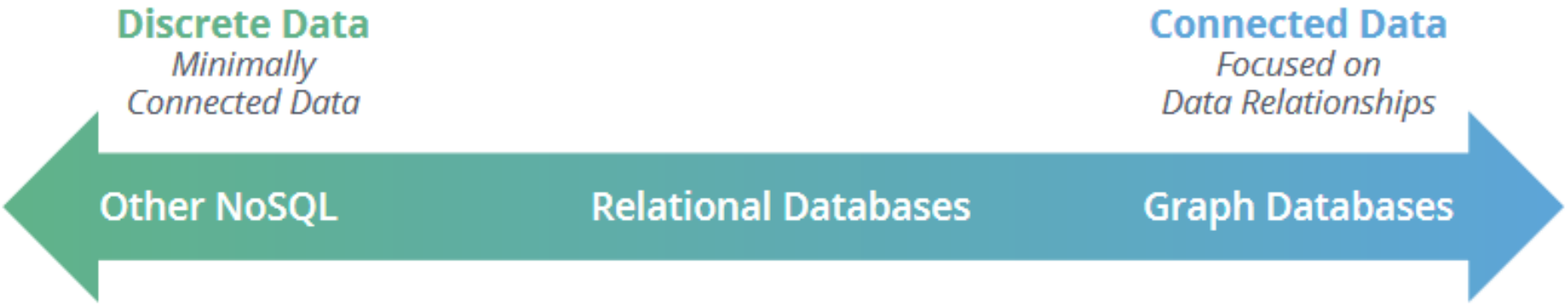




Many to Many Relationships

Why Graph Databases

- Leverage complex and dynamic relationships in highly connected data
- Store relationship data as first-class entities



Discrete versus connected data

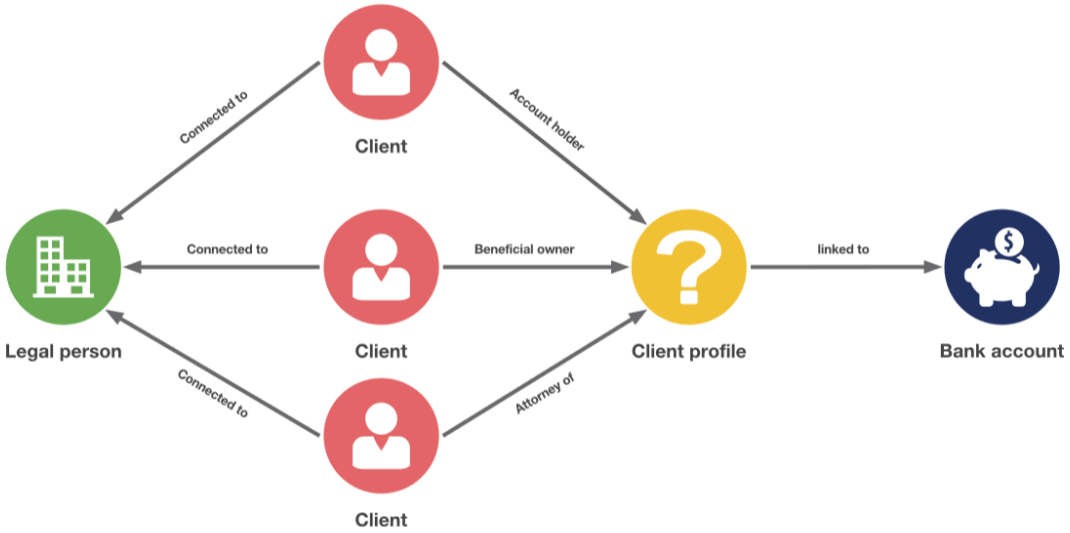
https://www.neo4j.com

Graph Database Use Cases

- Fraud detection: identify rings of fraudsters linked by shared attributes
- Recommendation engines: "users similar to you also purchased..."
- Knowledge graphs: connect entities and concepts for search and inference
- Network and IT operations: model infrastructure dependencies
- Identity and access management: complex role hierarchies and permissions

Use Case: Anti-Money Laundering

- International Consortium of Investigative Journalists (ICIJ)
- Panama Papers
  - 100,000 clients from 203 countries
- Graph Size
  - 275,000 nodes with 400,000 relationships

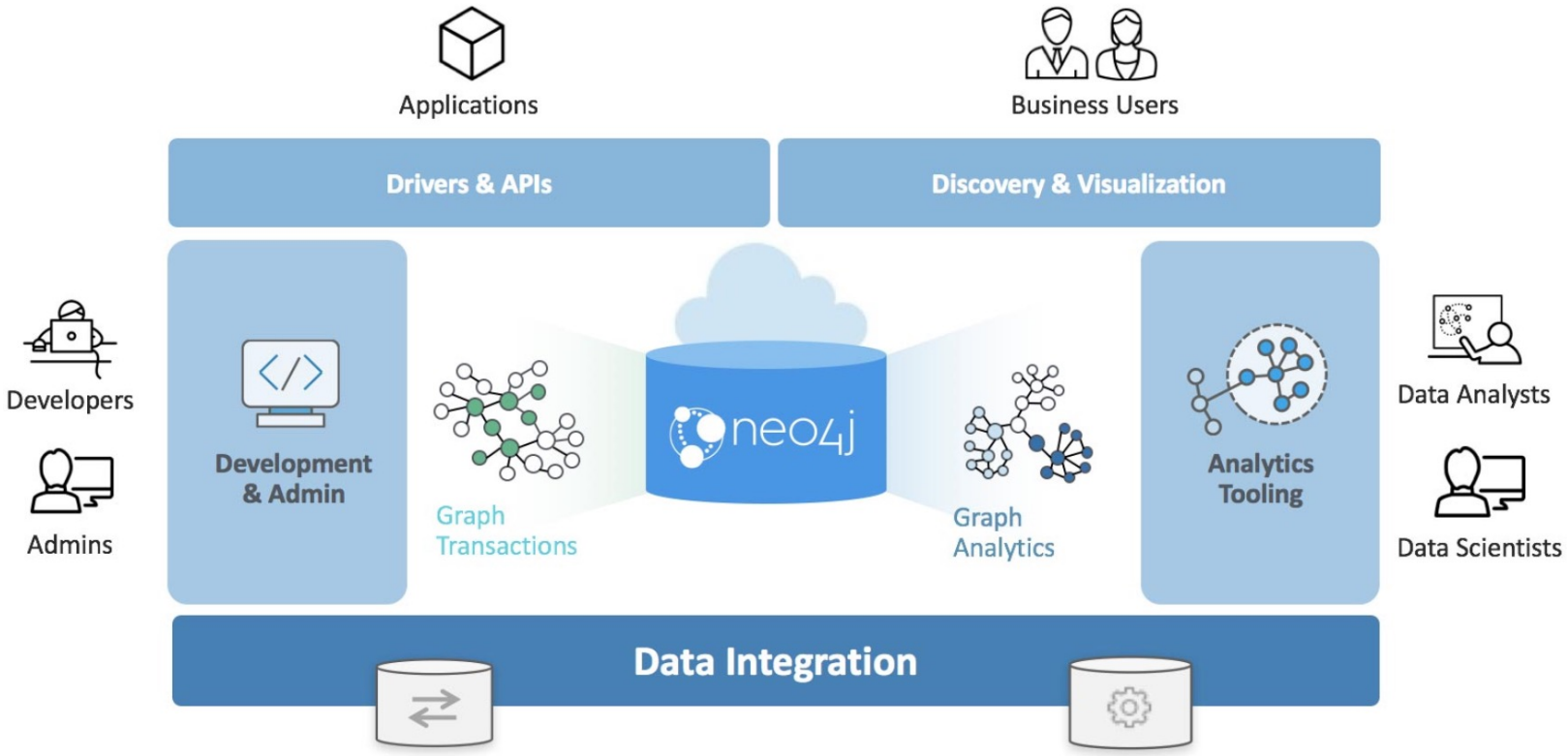


Schema representing the entities and connections within the Swiss Leaks dataset.

https://www.icij.org

https://go.neo4j.com/rs/710-RRC-335/images/Neo4j_CS_ICIJ_ENA4.pdf

Neo4j Architecture



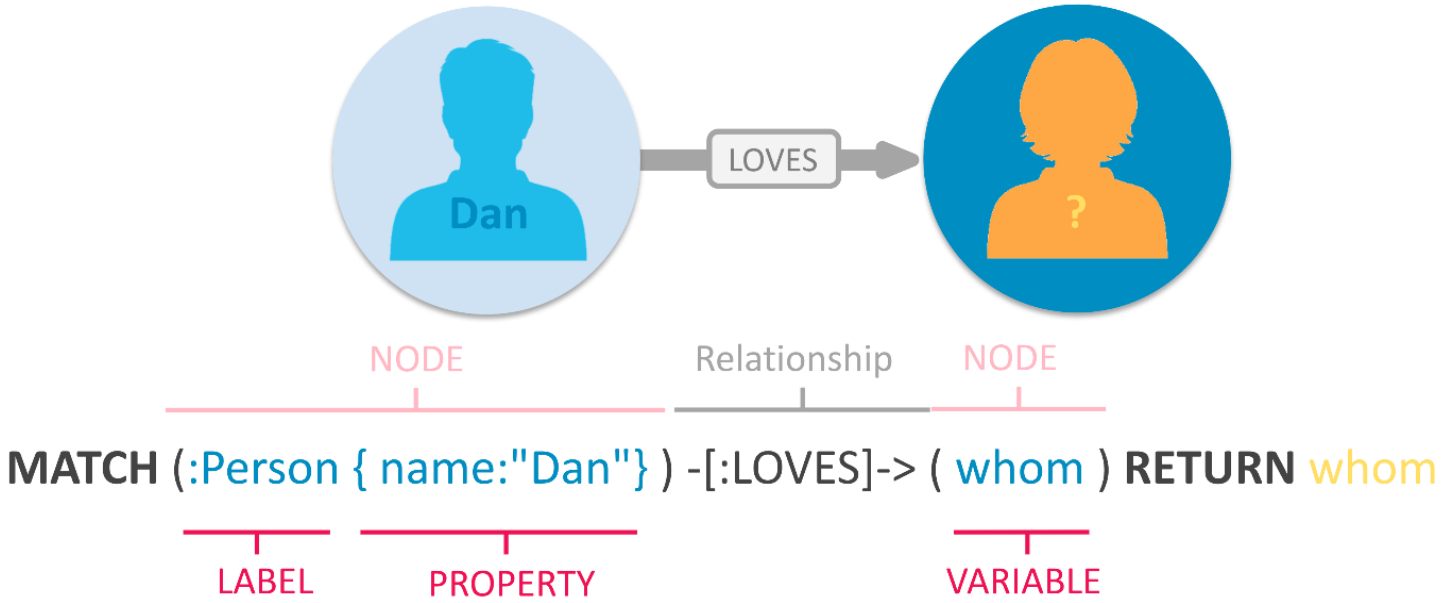
Neo4J Architecture

Cypher Query Language

- Cypher: declarative graph query language for Neo4j; visual, ASCII-art pattern syntax
- MATCH (n)-[:REL]->(m): describe the subgraph pattern to find
- CREATE / MERGE: add nodes and relationships; MERGE avoids duplicates
- RETURN, WHERE, ORDER BY, LIMIT: SQL-like result shaping
- Pattern variables: assign names to paths; use in WHERE or RETURN clauses
  - MATCH (p:Person)-[:ACTED_IN]->(m:Movie) WHERE m.released > 1990 RETURN p.name, m.title

Cypher Anatomy

- Open-source declarative query language
- Expressive and efficient querying and updating of graph data
- Focus on what to retrieve, and not how to retrieve

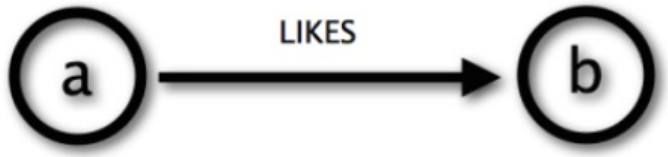


https://www.neo4j.com

Cypher Nodes and Relationships

- Nodes are surrounded with parentheses which look like circles
- Relationships are denoted by arrows between two nodes. Additional information can be placed in square brackets inside of the arrow

Cypher using relationship 'likes'



Cypher

```
(a) -[:LIKES]-> (b)
```

https://neo4j.com/developer/cypher-query-language/

Cypher General Syntax

Finding Nodes/Relationships	`MATCH (n1:Label1)-[rel:TYPE]->(n2:Label2)`
Constraints/Filters	`WHERE rel.property > {value}`
Formatting output	`RETURN rel.property, type(rel)`

https://neo4j.com/developer/cypher-query-language/

Friend-of-a-Friend Pattern

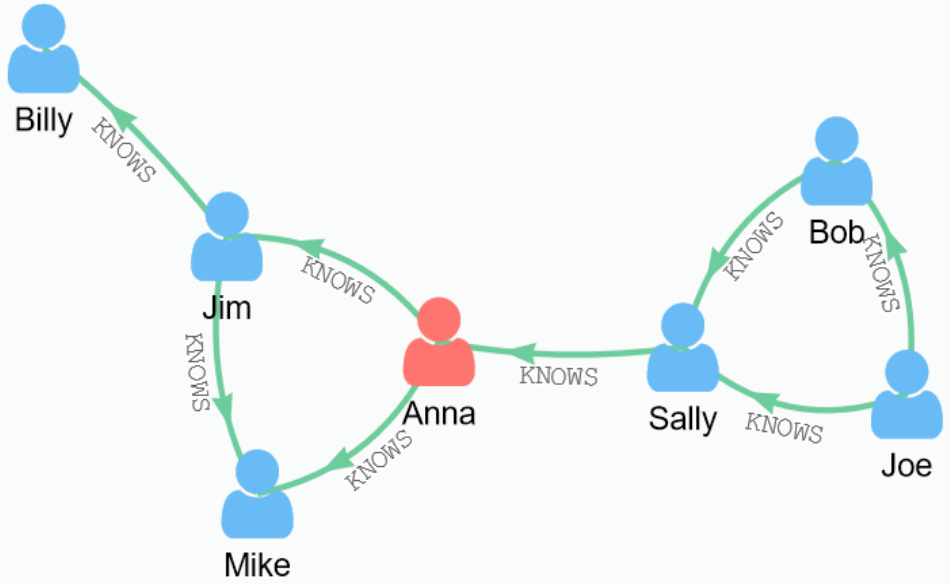
friend-of-a-friend `(user)-[:KNOWS]-(friend)-[:KNOWS]-(foaf)`

Friends of Friends

Find all of Joe's second-degree friends

```
MATCH
  (person:Person)-[:KNOWS]-(friend:Person)-[:KNOWS]-
  (foaf:Person)
WHERE
  person.name = "Joe"
  AND NOT (person)-[:KNOWS]-(foaf)
RETURN
  foaf
```

Joe knows Sally, and Sally knows Anna. Bob is excluded from the result because, in addition to being a 2nd-degree friend through Sally, he's also a first-degree friend.



Common Friends Pattern

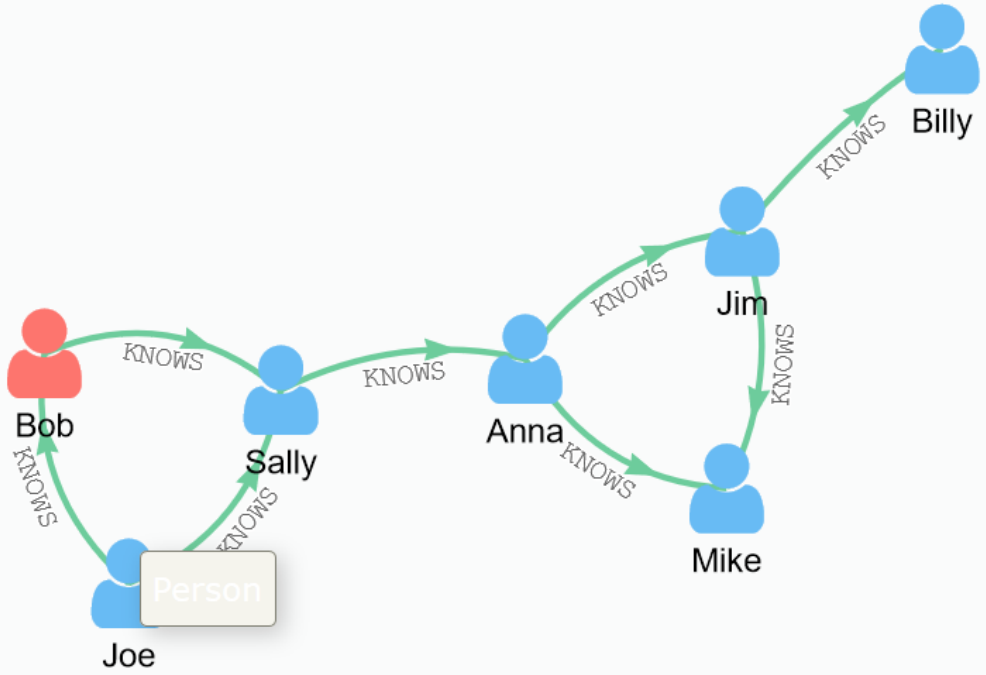
friend-of-a-friend `(user)-[:KNOWS]-(friend)-[:KNOWS]-(foaf)`

Common Friends

Find friends in common between Joe and Sally

```
MATCH
  (user:Person)-[:KNOWS]-(friend)-
    [:KNOWS]-(foaf:Person)
WHERE
  user.name = "Joe" AND foaf.name = "Sally"
RETURN
  friend.name AS friend
```

Joe and Sally both know Bob.



https://neo4j.com/developer/

Shortest Path Pattern

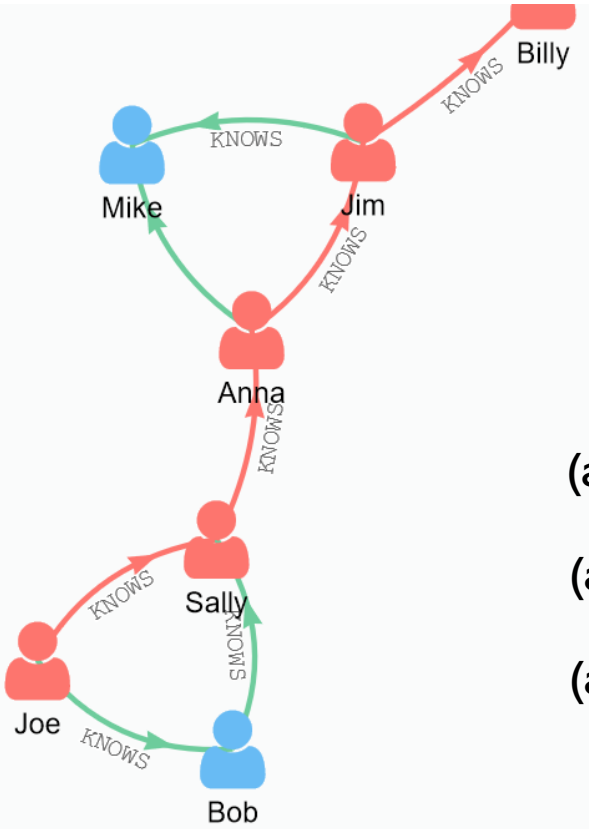
shortest path: `path = shortestPath( (user)-[:KNOWS*..5]-(other) )`

Connecting Paths

Find all paths, up to 6 degrees, between Joe and Billy

```
MATCH
  path = shortestPath(
    (p1:Person)-[:KNOWS*..6]-(p2:Person)
  )
WHERE
  p1.name = "Joe" AND p2.name = "Billy"
RETURN
  path
```

There's only one shortest path between Joe and Billy in this dataset, so that path is returned.



(a)-[*3..5]->(b)

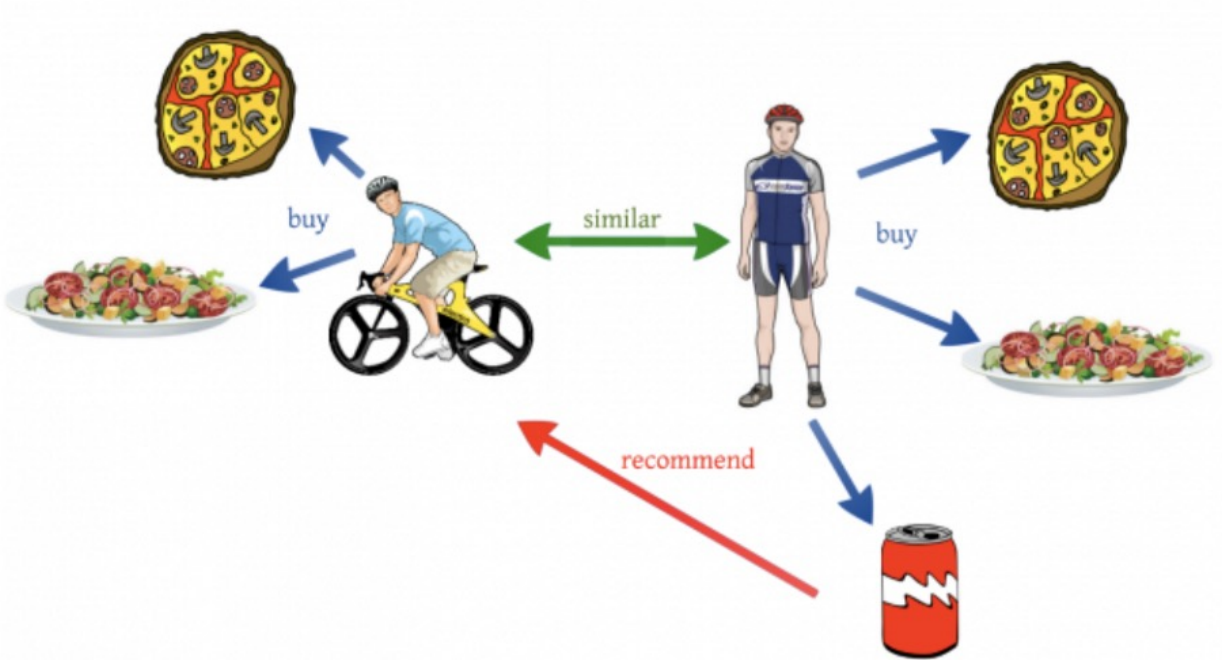
(a)-[*3..]->(b)

(a)-[*..5]->(b)

https://neo4j.com/developer/

Collaborative Filtering Pattern

collaborative filtering `(user)-[:PURCHASED]->(product)<-[:PURCHASED]-()-[:PURCHASED]->(otherProduct)`

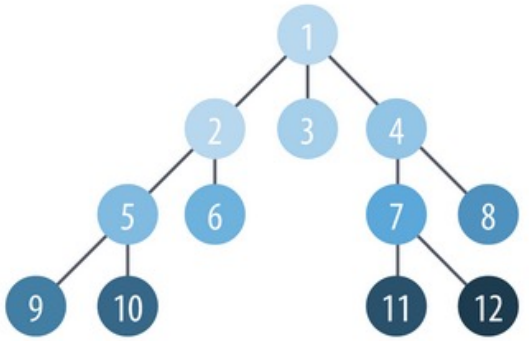


Graph Traversal Algorithms

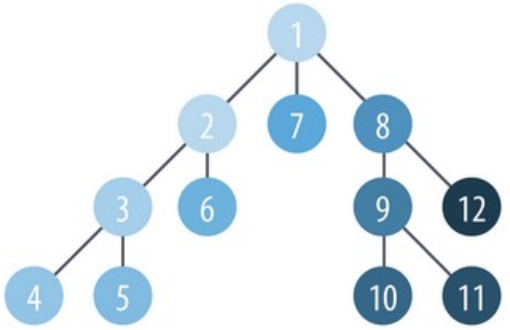
- Breadth-First Search (BFS): explore nearest neighbors first; shortest path in unweighted graphs
- Depth-First Search (DFS): follow one path as deep as possible before backtracking
- Dijkstra / A*: shortest weighted path; GPS routing, network optimization
- PageRank: rank nodes by number and quality of incoming edges
- Community detection: identify clusters of densely connected nodes

Breadth-First and Depth-First Search

Algorithm	Description	Example Usage
Breadth First Search	Traverses a tree structure by fanning out to explore the nearest neighbors and then their sublevel neighbors	Locating neighbor nodes in GPS systems to identify nearby places of interest
Depth First Search	Traverses a tree structure by exploring as far as possible down each branch before backtracking	Discovering an optimal solution path in gaming simulations with hierarchical choices



Breadth First Search

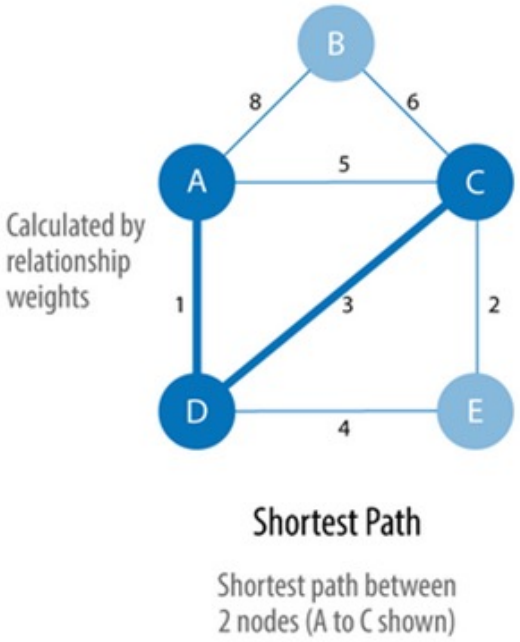


Depth First Search
Walks down each branch first

Graph Algorithms - Neo4j version

Shortest Path Algorithms

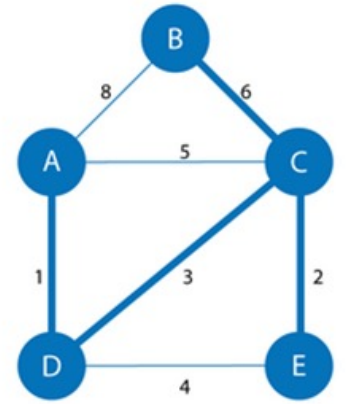
Algorithm	Description	Example Usage
Shortest Path	Shortest path between a pair of nodes	Driving directions between two locations
All Pairs Shortest Path	Shortest path between all pairs of nodes in the graph	Evaluating alternate traffic routes
Single Source Shortest Path	Shortest path between a single root node and all other nodes	Least cost routing of phone calls



Graph Algorithms - Neo4j version

Spanning Tree and Random Walk

Algorithm	Description	Example Usage
Minimum Spanning Tree	Calculates the path in a connected tree structure with the smallest cost for visiting all nodes	Optimizing connected routing, such as laying cable or garbage collection
Random Walk	Returns a list of nodes along a path of specified size by randomly choosing relationships to traverse.	Augmenting training for machine learning or data for graph algorithms.



Minimum Spanning Tree

Shortest path **connecting all nodes**
(A start shown)

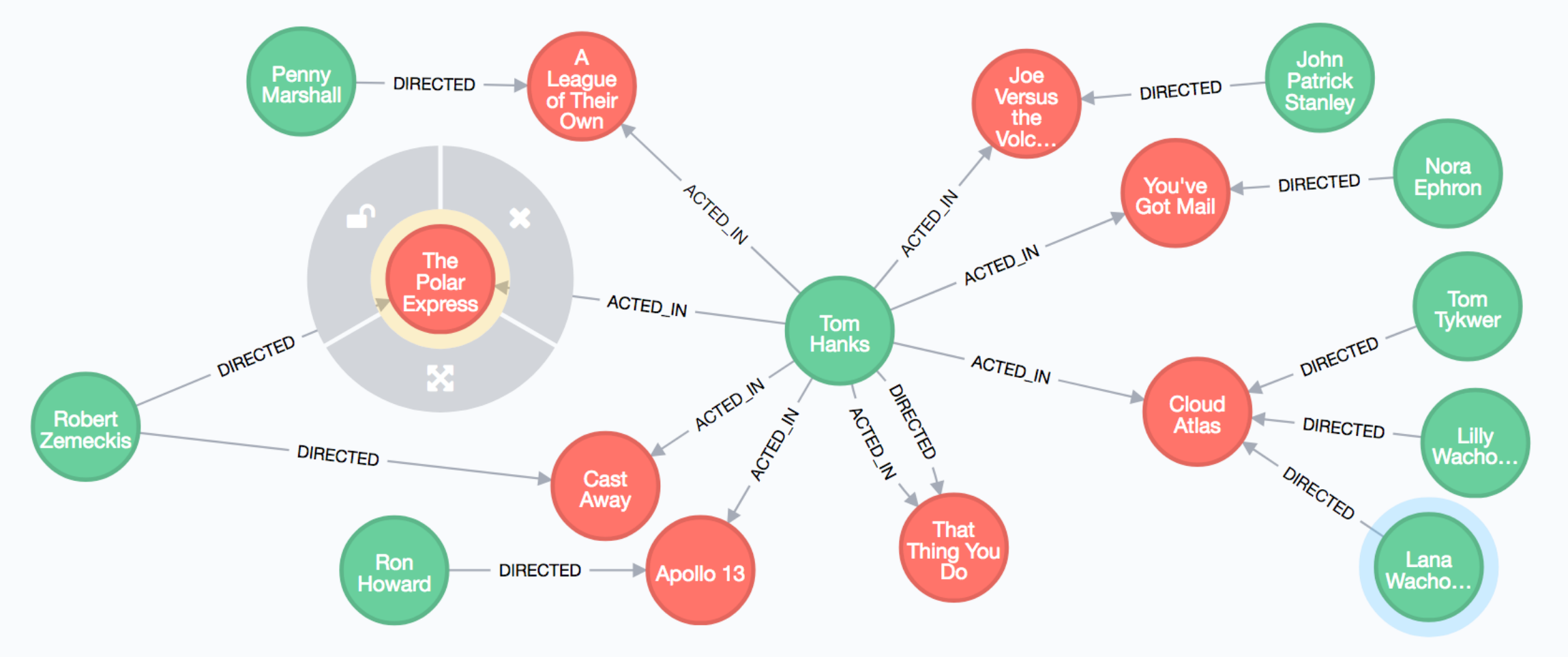
Traverses to the next unvisited node via the lowest weight **from any visited node**

Graph Algorithms - Neo4j version

Neo4j Movies Graph Lab

- Graph: Person nodes + Movie nodes connected by typed relationships
- Relationship types: [:ACTED_IN {roles:[…]}], [:DIRECTED], [:WROTE], [:PRODUCED], [:REVIEWED]
- Neo4j-MoviesDB-DDL.cypher — seed data:
  - CREATE (Keanu:Person {name:'Keanu Reeves', born:1964})
  - CREATE (Keanu)-[:ACTED_IN {roles:['Neo']}]->(TheMatrix)
- Sample queries:
  - MATCH (p:Person)-[:ACTED_IN]->(m:Movie) WHERE m.title='The Matrix' RETURN p.name
  - MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d) RETURN a.name, d.name, m.title
- Neo4j-Friends.ipynb: FOLLOWS graph with py2neo — mutual friends, shortest path traversal
  - MATCH p=shortestPath((a:Person)-[:FOLLOWS*]-(b:Person)) RETURN length(p)

Movies Graph

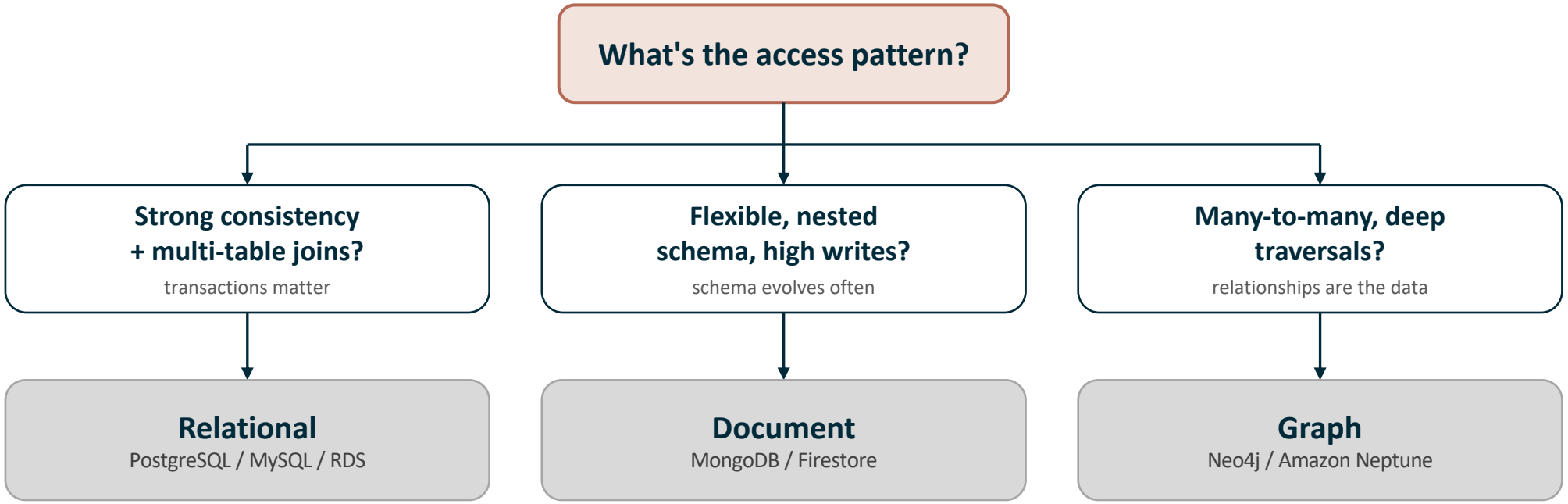




Access Patterns and Trade-offs

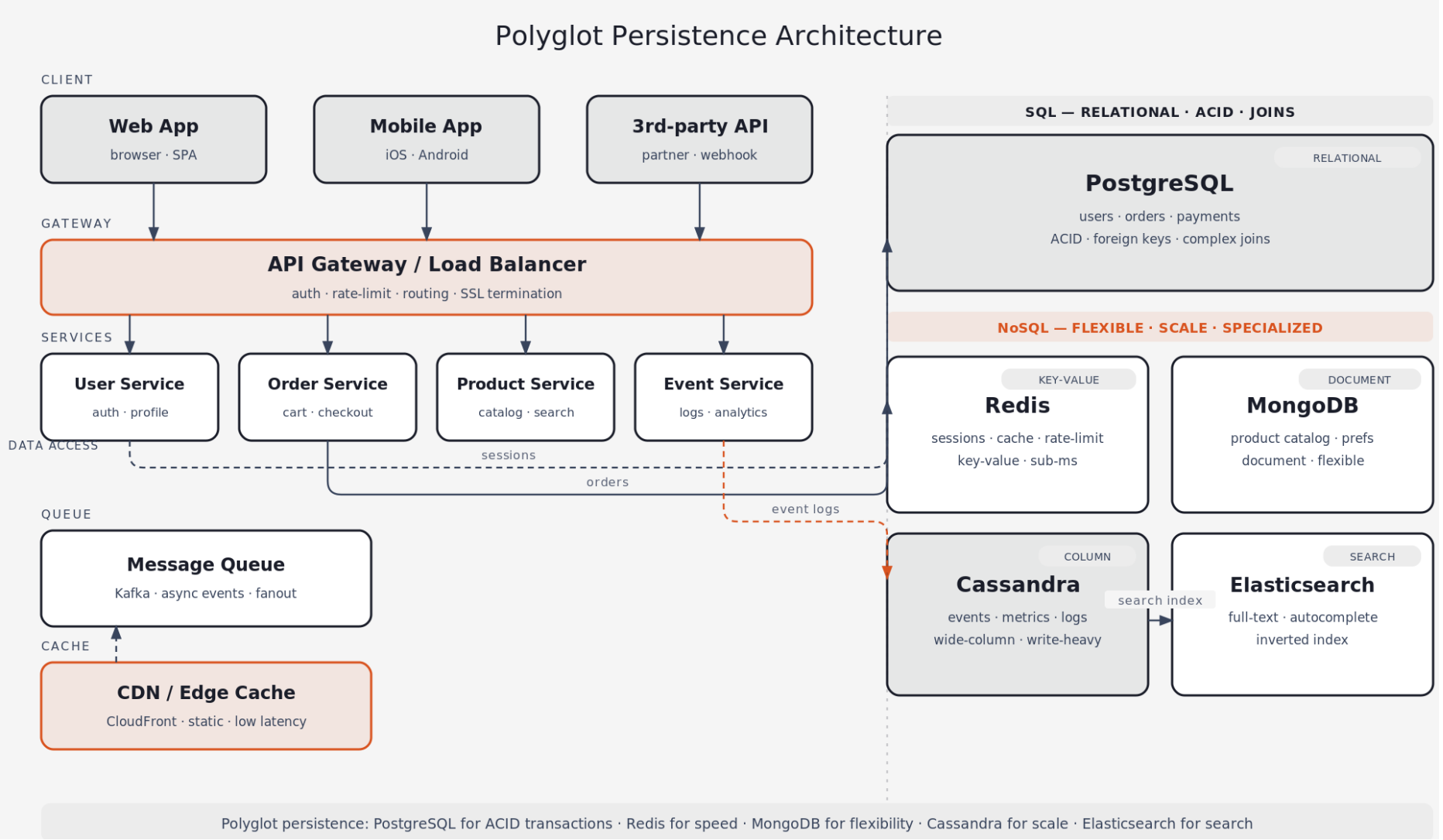
Module 5

Relational vs Document vs Graph



MOST REAL SYSTEMS ANSWER "YES" TO MORE THAN ONE — HENCE POLYGLOT PERSISTENCE

Polyglot Persistence



Model Comparison

Dimension	Relational	DynamoDB	MongoDB	Neo4j
Consistency	Strong (ACID)	Eventual by default; strong optional	Strong on primary; tunable	ACID; causal in clusters
Data modeling	Normalized; joins	Denormalized by access pattern	Embed or reference	Nodes and relationships
Query shape	SQL joins	Key lookups; avoid scans	find() and aggregation	Cypher pattern match
Indexing	B-tree indexes	GSI and LSI	Single, compound, text, geo	Label and property indexes
Scaling	Mostly vertical	Serverless, automatic	Sharding	Less horizontal scaling

Summary

- NoSQL trades strict consistency for scale and schema flexibility.
- CAP theorem: pick two of consistency, availability, partition tolerance.
- Document databases use embedding for speed, referencing for shared data.
- Key-value stores (DynamoDB) are built around keys: query by partition key, avoid scans.
- Graph databases treat relationships as first-class citizens.
- NoSQL modeling starts from access patterns, not entities — design keys/documents/relationships around the queries you'll actually run.
- Polyglot persistence: use the right database for each workload.